

Documentazione TIW

Configuratore di prodotto

Un prodotto composto definisce un articolo configurabile come una struttura gerarchica comprendente diverse parti. Ad esempio, un PC configurabile comprende sistema di elaborazione, sistema di memoria, sistema di archiviazione, sistema di raffreddamento, case e alimentazione. Il sistema di elaborazione comprende scheda madre, CPU, memoria RAM e scheda grafica. Il sistema di memoria comprende primo disco fisso, secondo disco fisso, unità RAID, unità DVD.

Un prodotto composto ha un codice, un nome, una descrizione e una fascia di prezzo (prezzo minimo e prezzo massimo). Un prodotto composto è associato a uno o più sottoprodotti (si veda l'esempio riportato in precedenza del PC configurabile). I sottoprodotti di un prodotto composto possono essere prodotti composti o semplici.

Un prodotto semplice ha un codice, un nome ed è associato ai diversi SKU (Stock Keeping Unit) che lo possono realizzare (ad esempio, la CPU può essere associata alle SKU AMD Ryzen 5 9500F 6 core, AMD Ryzen 7 7700X 8 core, ...).

Ogni SKU ha un codice (numero intero), un nome, una fotografia, una descrizione tecnica e un prezzo. I codici dei prodotti sono numeri interi unici.

Il fornitore crea le SKU e definisce il modello gerarchico di un prodotto composto. Crea il prodotto composto di più alto livello, aggiunge i sottoprodotti composti e semplici e associa ai prodotti semplici le SKU corrispondenti. Il cliente definisce la configurazione di prodotto che desidera acquistare a partire da un prodotto composto. Seleziona la SKU desiderata per ogni prodotto semplice facente parte del prodotto composto.

Configuratore di prodotto

Un prodotto composto definisce un articolo configurabile come una struttura gerarchica comprendente diverse parti. Ad esempio, un PC configurabile comprende sistema di elaborazione, sistema di memoria, sistema di archiviazione, sistema di raffreddamento, case e alimentazione. Il sistema di elaborazione comprende scheda madre, CPU, memoria RAM e scheda grafica. Il sistema di memoria comprende primo disco fisso, secondo disco fisso, unità RAID, unità DVD.

Un **prodotto composto** ha un **codice**, un **nome**, una **descrizione** e una **fascia di prezzo** (prezzo minimo e prezzo massimo). Un prodotto composto è **associato a uno o più sottoprodotti** (si veda l'esempio riportato in precedenza del PC configurabile). I sottoprodotti di un prodotto composto possono essere prodotti composti o semplici.

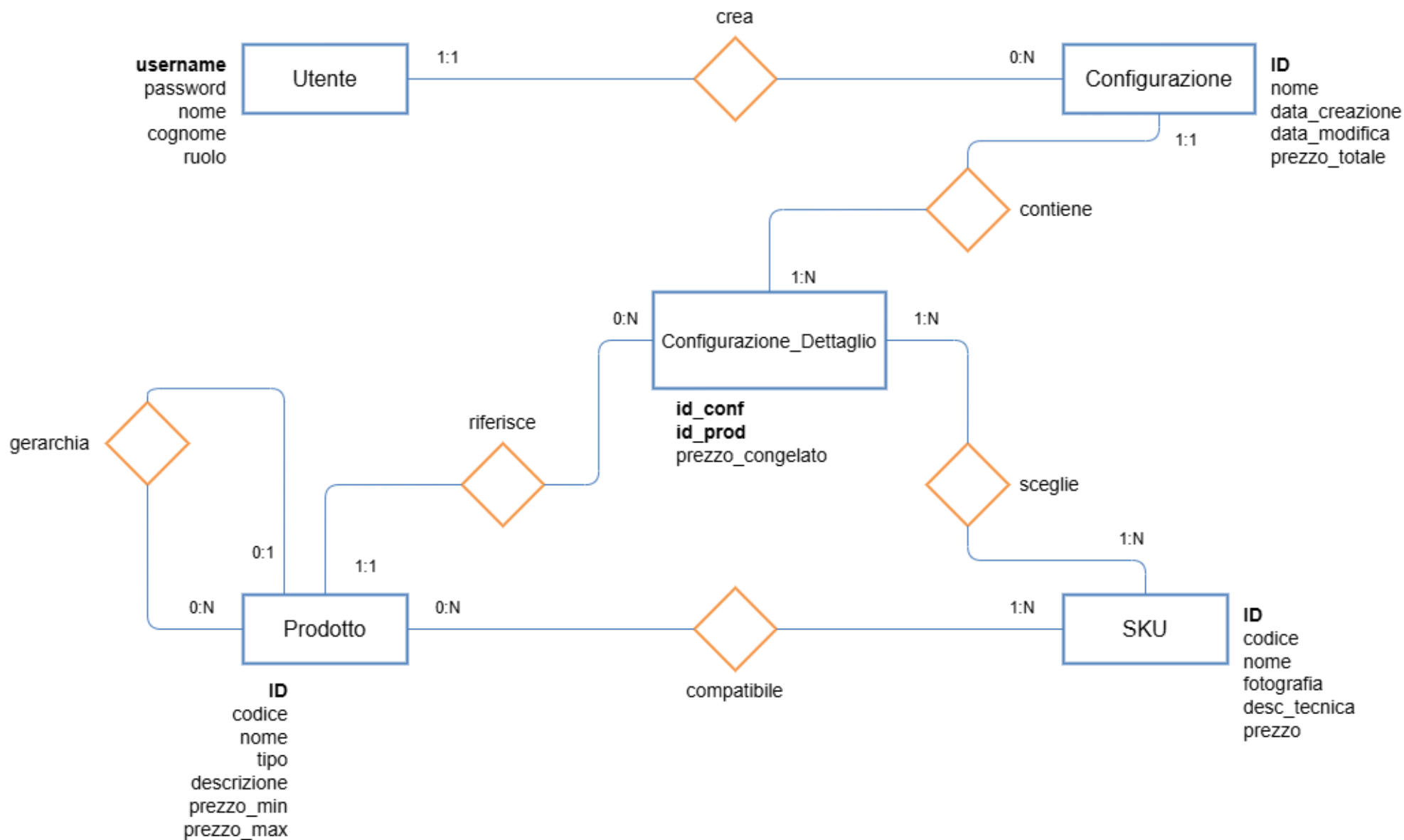
Un **prodotto semplice** ha un **codice**, un **nome** ed è **associato** ai diversi SKU (Stock Keeping Unit) che lo possono realizzare (ad esempio, la CPU può essere associata alle SKU AMD Ryzen 5 9500F 6 core, AMD Ryzen 7 7700X 8 core, ...).

Ogni **SKU** ha un **codice** (numero intero), un **nome**, una **fotografia**, una **descrizione tecnica** e un **prezzo**. I codici dei prodotti sono numeri interi unici.

Il **fornitore** crea le SKU e definisce il modello gerarchico di un prodotto composto. Crea il prodotto composto di più alto livello, aggiunge i sottoprodotti composti e semplici e associa ai prodotti semplici le SKU corrispondenti. Il **cliente** **definisce** la **configurazione** di prodotto che desidera acquistare a partire da un prodotto composto. **Seleziona** la SKU desiderata per ogni prodotto semplice facente parte del prodotto composto.

Entities, attributes, relationships

Database design



Local database schema

```
CREATE TABLE utente (  
  username          VARCHAR(50)  PRIMARY  
KEY,  
  password_hash     VARCHAR(255) NOT  
NULL,  
  nome              VARCHAR(100) NOT  
NULL,  
  cognome           VARCHAR(100) NOT  
NULL,  
  ruolo             ENUM('FORNITORE', 'CLIENTE')  
NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE sku (  
  id                INT AUTO_INCREMENT PRIMARY KEY,  
  codice            INT      NOT NULL UNIQUE,  
  nome              VARCHAR(200) NOT NULL,  
  fotografia        VARCHAR(500) NULL,  
  descrizione_tecnica TEXT      NULL,  
  prezzo            DECIMAL(10, 2) NOT NULL,  
  
  CONSTRAINT chk_sku_prezzo_pos CHECK (prezzo  
>= 0)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;  
  
CREATE INDEX idx_sku_nome ON sku(nome);
```

Local database schema

```
CREATE TABLE prodotto (  
  id                INT                AUTO_INCREMENT PRIMARY KEY,  
  codice            INT                NOT NULL UNIQUE,  
  nome              VARCHAR(200)       NOT NULL,  
  tipo              ENUM('SEMPLICE', 'COMPOSTO') NOT NULL,  
  descrizione       TEXT              NULL,  
  prezzo_min        DECIMAL(10, 2)     NULL,  
  prezzo_max        DECIMAL(10, 2)     NULL,  
  id_padre          INT              NULL,  
  CONSTRAINT chk_prezzo_min_pos CHECK (prezzo_min IS NULL OR prezzo_min >= 0),  
  CONSTRAINT chk_prezzo_max_pos CHECK (prezzo_max IS NULL OR prezzo_max >= 0),  
  CONSTRAINT chk_fascia_prezzo CHECK (  
    prezzo_min IS NULL OR prezzo_max IS NULL OR prezzo_max >= prezzo_min  
  ),  
  
  CONSTRAINT fk_prodotto_padre FOREIGN KEY (id_padre) REFERENCES prodotto(id)  
  ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
CREATE INDEX idx_prodotto_id_padre ON prodotto(id_padre);  
CREATE INDEX idx_prodotto_nome    ON prodotto(nome);
```

Local database schema

```
CREATE TABLE prodotto_sku (  
  id_prodotto      INT      NOT NULL,  
  id_sku           INT      NOT NULL,  
  
  PRIMARY KEY (id_prodotto, id_sku),  
  CONSTRAINT fk_ps_prodotto FOREIGN KEY (id_prodotto) REFERENCES prodotto(id)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_ps_sku FOREIGN KEY (id_sku) REFERENCES sku(id)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Local database schema

```
CREATE TABLE configurazione (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  cliente_username VARCHAR(50) NOT NULL,  
  prodotto_radice_id INT NOT NULL,  
  nome VARCHAR(200) NOT NULL,  
  data_creazione DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  data_modifica DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  prezzo_totale DECIMAL(10, 2) NOT NULL DEFAULT 0.00,  
  
  CONSTRAINT chk_cfg_prezzo_pos CHECK (prezzo_totale >= 0),  
  
  CONSTRAINT fk_cfg_cliente FOREIGN KEY (cliente_username) REFERENCES utente(username)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_cfg_prodotto_radice FOREIGN KEY (prodotto_radice_id) REFERENCES prodotto(id)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
  
CREATE INDEX idx_cfg_cliente ON configurazione(cliente_username);  
CREATE INDEX idx_cfg_data ON configurazione(data_creazione DESC);
```

Local database schema

```
CREATE TABLE configurazione_dettaglio (  
  id_configurazione          INT                NOT NULL,  
  id_prodotto                INT                NOT NULL,  
  id_sku                     INT                NOT NULL,  
  prezzo_unitario_congelato  DECIMAL(10, 2)      NOT NULL,  
  
  PRIMARY KEY (id_configurazione, id_prodotto),  
  
  CONSTRAINT fk_cd_configurazione FOREIGN KEY (id_configurazione) REFERENCES configurazione(id)  
    ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT fk_cd_prodotto FOREIGN KEY (id_prodotto) REFERENCES prodotto(id)  
    ON DELETE RESTRICT ON UPDATE CASCADE,  
  CONSTRAINT fk_cd_sku FOREIGN KEY (id_sku) REFERENCES sku(id)  
    ON DELETE RESTRICT ON UPDATE CASCADE,  
  
  CONSTRAINT chk_cd_prezzo_pos CHECK (prezzo_unitario_congelato >= 0)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Application requirements analysis (fornitore)

A seguito del **login**, il fornitore accede a una pagina **HOME FORNITORE** che contiene una **form per creare una SKU**, una **form per creare un prodotto composto** e una **form per creare un prodotto semplice**.

- Con la form per creare SKU il fornitore inserisce il valore degli attributi (codice, nome, fotografia, descrizione tecnica e un prezzo). Un **bottone SALVA** permette di **inserire la SKU nella base di dati permanentemente**.
- Con la form per creare un prodotto semplice il fornitore inserisce il valore degli attributi (codice e nome) e associa il prodotto semplice alle SKU che lo realizzano, mediante la spunta di **una o più check box** da un elenco delle SKU. Tale elenco è ordinato per codice delle SKU decrescente. Un **bottone SALVA** permette di **inserire il prodotto semplice e le associazioni alle SKU nella base di dati permanentemente**.
- Con la form per creare un prodotto composto il fornitore inserisce il valore degli attributi (codice, nome, descrizione e fascia di prezzo) e definisce i sottoprodotti selezionando prodotti semplici o composti, precedentemente creati ma non ancora utilizzati in altri prodotti. L'associazione dei sottoprodotti avviene mediante la spunta di **una o più check box** in un elenco dei prodotti (semplici e composti). Tale elenco è ordinato per nome decrescente (case-sensitive) del prodotto. Un **bottone SALVA** permette di **inserire il prodotto composto e le associazioni ai sottoprodotti nella base di dati permanentemente**.
- La profondità massima della gerarchia dei prodotti è 3 livelli sotto il prodotto di primo livello.

Pages (views), **view components**, **events**, **actions**

Application requirements analysis (fornitore)

A seguito del salvataggio di un oggetto (SKU, prodotto semplice o composto) la pagina **HOME FORNITORE** mostra i dati seguenti:

- Se si tratta di una SKU i soli **valori degli attributi della nuova SKU**.
- Se si tratta di un prodotto semplice, **i valori degli attributi e la lista delle SKU che lo realizzano**.
- Se si tratta di un prodotto composto, **i valori degli attributi e una lista nidificata multilivello che mostra ricorsivamente tutti i componenti del prodotto composto e le SKU associate con i prodotti semplici**.
- La **modifica** di un prodotto semplice o composto avviene mediante **cancellazione** del prodotto e successiva **creazione** di un nuovo prodotto con ugual codice e nome e riassegnazione di sottoprodotti o SKU.

Una pagina **RICERCA PRODOTTI** contiene una **form per la ricerca (case-insensitive) di prodotti e SKU**. La pagina contiene **un solo campo di input e il bottone RICERCA**. L'applicazione **verifica che il campo di input non sia vuoto**. La **pressione del bottone RICERCA** comporta la **ricerca (case-insensitive)** applicata ai campi nome e descrizione dei prodotti e ai campi nome e descrizione tecnica delle SKU.

Pages (views), view components, events, actions

Application requirements analysis (fornitore)

I risultati della ricerca sono presentati come un **elenco cliccabile**. Il **click su un elemento dell'elenco** comporta **l'apparizione di tutti gli attributi e di tutte le associazioni dell'oggetto** (SKU, prodotto semplice o composto) con le stesse modalità descritte per la presentazione di un oggetto appena creato. Tale visualizzazione comprende anche i **link RIMUOVI ed ELIMINA** che agiscono nel modo seguente.

- Per un prodotto semplice, nella lista delle SKU ogni SKU ha un **link RIMUOVI** che **elimina la relazione tra il prodotto semplice e la SKU**. **La pressione di tale link** comporta **l'aggiornamento della base di dati, il ricarico della pagina HOME FORNITORE e l'aggiornamento dei dati**.
- Per un prodotto, nella lista nidificata ogni sottoprodotto ha un **link RIMUOVI** che **elimina la relazione tra il sottoprodotto e il prodotto composto che lo contiene**. **La pressione di tale link** comporta **l'aggiornamento della base di dati, il ricarico della pagina HOME FORNITORE e l'aggiornamento dei dati**.
- Per ogni oggetto (prodotto composto, prodotto semplice e SKU) compare un **link ELIMINA**. **La pressione di tale link** comporta **l'eliminazione dell'oggetto e dei suoi eventuali sottoprodotti e SKU dalla base di dati, il ricarico della pagina HOME FORNITORE e l'aggiornamento dei dati**. L'eliminazione deve anche **cancellare dalla base di dati tutte le associazioni con i prodotti di cui l'oggetto eliminato (prodotto o SKU) fa eventualmente parte**.

Application requirements analysis (cliente)

A seguito del **login**, il cliente accede a una pagina **HOME CLIENTE** che **elenca tutti i prodotti composti di primo livello** in ordine alfabetico di nome decrescente. Se i prodotti sono più di 10, la pagina mostra inizialmente i primi 10 e **i bottoni PRECEDENTI e SUCCESSIVI** permettono di **scorrere l'elenco visualizzando 10 prodotti per volta**. Tali bottoni devono comparire solo se esiste un insieme precedente o successivo di prodotti. **Selezionando un prodotto di primo livello** compare una nuova pagina **SCELTA SKU** che riporta il **prodotto di primo livello selezionato e al di sotto di questo l'elenco dei prodotti di secondo livello eventualmente presenti**. **Selezionando un prodotto di secondo livello** compare al di sotto di questo **l'elenco dei prodotti di terzo livello** eventualmente presenti, e così via.

A fianco di ogni prodotto semplice, a qualunque livello esso si collochi, compare un **menu a tendina** che permette di scegliere **la SKU per quel prodotto**. Una volta selezionate tutte le SKU necessarie per la configurazione il cliente può **usare** un **bottone SALVA** per **memorizzare la propria configurazione nella base di dati**. Il salvataggio di una configurazione richiede l'inserimento di un **nome per la configurazione**. La configurazione è associata al cliente che l'ha creata, alla data di creazione e al **prezzo totale calcolato** come somma dei costi delle SKU selezionate.

Se il cliente preme il bottone SALVA ma alcune SKU non sono state scelte l'applicazione **notifica un errore e ripropone la pagina SCELTA SKU con le selezioni fatte in precedenza**. Se il salvataggio va a buon fine l'applicazione mostra una pagina **DETTAGLIO CONFIGURAZIONE** che mostra **le SKU selezionate con i rispettivi dati e il prezzo totale della configurazione**.

Una pagina **LE MIE CONFIGURAZIONI** mostra **tutte le configurazioni create dall'utente** in ordine di data di creazione decrescente e permette di **cancellare** una configurazione e di **clonare una configurazione**. **Un link MODIFICA** accanto a una configurazione permette di **aprire la pagina SCELTA SKU per quella configurazione**, di **modificare** la selezione delle SKU e di **salvare** la versione modificata della configurazione, **aggiornando la data di ultima modifica**.

Pages (views), view components, events, actions

Application requirements analysis (fornitore)

Estensioni versione JavaScript

La versione JavaScript estende e/o modifica le specifiche dell'interfaccia come segue:

- L'interfaccia deve essere realizzata in **una sola pagina**. L'invio di richieste al server è gestito in modo asincrono e provoca l'aggiornamento parziale dell'interfaccia limitatamente ai soli dati che variano.
- La creazione di una SKU avviene tramite una **form**. **A seguito della pressione del bottone SALVA SKU, la pagina mostra i dati della nuova SKU. Il click sul valore di un attributo della SKU rende il valore editabile** mediante **un campo di input**. **Il nuovo valore è salvato nella base di dati** automaticamente **a seguito dell'evento di uscita del mouse dal campo di input**.
- La creazione e la modifica di un prodotto composto si svolgono come segue. L'utente crea il prodotto di primo livello mediante una **form** in cui inserisce il valore degli attributi. **A seguito della pressione del bottone SALVA PRODOTTO** e alla creazione a lato client del prodotto di primo livello, **la pagina ne mostra i dati. Il click sul valore di un attributo del prodotto di primo livello rende il valore editabile** mediante un **campo di input**. Il nuovo valore è **salvato nella base di dati** mediante **la pressione** del **bottone SALVA PRODOTTO**.
- La creazione e la modifica di un prodotto composto devono avvenire inizialmente solo nel client. Quando l'inserimento di tutti i dati è completo, un **bottone SALVA PRODOTTO** permette la **registrazione permanente del prodotto composto nella base di dati**. Il tentativo di **creare un prodotto composto mancante di SKU** per qualcuno dei prodotti semplici **produce un messaggio errore**, ma non la perdita dei dati inseriti a lato client fino a quel momento.

Pages (views), view components, events, actions

Application requirements analysis (fornitore)

Estensioni versione JavaScript

A fianco del prodotto compaiono **un bottone +, un bottone - e un bottone -***.

- Il **bottone +** accanto a un prodotto composto **produce la visualizzazione di un menu con le scelte “sottoprodotto composto” e “sottoprodotto semplice”**. La scelta di **“sottoprodotto composto”** comporta **la creazione di un prodotto composto come sottoelemento dell’elemento corrente**. La scelta di **“sottoprodotto semplice”** comporta **la creazione di un prodotto semplice come sottoelemento dell’elemento corrente**. La creazione di un prodotto semplice comporta l’inserimento di codice e nome in **campi di input** e la scelta delle SKU da un **menu a selezione multipla**. In entrambi i casi, il sottoprodotto viene mostrato sotto il prodotto di cui fa parte in modo da formare una visualizzazione a lista nidificata. Il **click sul valore di un attributo** di un prodotto semplice o composto nidificato **rende il valore editabile** mediante un **campo di input**. La modifica è **salvata nella base di dati mediante la pressione del bottone SALVA PRODOTTO**
- Il **bottone +** accanto a un prodotto semplice **produce la visualizzazione di un menu con le scelte “Nuova SKU” e “SKU esistente”**. La scelta di **“nuova SKU”** comporta l’apparizione di una **form per la creazione di una nuova SKU**. La scelta di **“SKU esistente”** comporta l’apparizione di una **form per la scelta di una o più SKU da un elenco a scelta multipla di tutte le SKU esistenti**. In entrambi i casi, le SKU associate sono mostrate sotto il prodotto semplice in modo da formare una **visualizzazione a lista nidificata**. La modifica è **salvata nella base di dati** mediante **la pressione** del **bottone SALVA PRODOTTO**
- Il **bottone “-”** a fianco di un prodotto appare solo quando esiste un prodotto padre. Esso **produce la cancellazione della relazione tra il sottoprodotto e il prodotto padre, ma lascia intatto il prodotto figlio nella base di dati**. La modifica è **salvata nella base di dati** mediante **la pressione** del **bottone SALVA PRODOTTO**. Il **prodotto figlio diventa un prodotto di primo livello**.

Pages (views), view components, events, actions

Application requirements analysis (fornitore)

Estensioni versione JavaScript

- Il **bottone** **-*** a fianco di un prodotto **produce la cancellazione del prodotto stesso e dei suoi sottoprodotti ricorsivamente**. Le **SKU associate vengono eliminate solo se non risultano associate ad altri prodotti semplici**. La modifica è **salvata nella base di dati** mediante **la pressione** del **bottone SALVA PRODOTTO**
- Il **bottone** **-** a fianco di una SKU **produce la cancellazione della relazione tra SKU e prodotto**.
- Il **bottone** **-*** a fianco di una SKU **produce la sua cancellazione dalla base di dati**. La modifica è **salvata nella base di dati** mediante **la pressione** del **bottone SALVA PRODOTTO**.

La funzione della **pagina RICERCA PRODOTTI** è realizzata all'interno della **singola pagina** dell'applicazione. **Dopo la visualizzazione** è possibile **modificare gli attributi dei prodotti e delle SKU visualizzati, eliminare associazioni e eliminare oggetti dalla base di dati** con le stesse modalità disponibili durante la creazione di un prodotto composto.

Undo/redo delle operazioni del fornitore lato client: l'inserimento a lato client (prima dell'aggiornamento della base di dati) dei dati di un prodotto composto consente le operazioni di undo e di redo. Le azioni soggette ad **undo/redo** sono: la **creazione di un prodotto composto, l'inserimento del valore degli attributi di un prodotto composto, la creazione di un sottoprodotto di un prodotto, la scelta di una SKU per un prodotto semplice, la cancellazione di un'associazione tra prodotti o tra prodotti e SKU**.

Pages (views), view components, events, actions

Application requirements analysis (cliente)

Estensioni versione JavaScript

L'interfaccia deve essere realizzata in **una sola pagina**. L'invio di richieste al server è gestito in modo asincrono e provoca l'aggiornamento **parziale dell'interfaccia** limitatamente ai soli dati che variano.

Pages (views), **view components**, **events**, **actions**

Completamento delle specifiche

Generali

- La pagina di default è la form di login
- Tutti i campi di ogni form sono obbligatori, salvo dove diversamente specificato
- Ogni pagina (eccetto login) mostra un link per tornare alla pagina HOME e un link di logout
- I codici di prodotti e SKU sono numeri interi e devono essere univoci nell'intero catalogo
- Il prezzo di una SKU non può essere negativo

Vincoli sul catalogo (verificati lato server)

- La profondità massima della gerarchia è verificata lato server prima di ogni inserimento di un figlio e non viene permessa lato client
- L'assenza di cicli nella gerarchia è verificata lato server prima di ogni inserimento di un figlio
- Un prodotto semplice o composto può avere al più un prodotto padre (unicità del padre garantita)
- Un prodotto semplice deve avere almeno una SKU associata per poter essere incluso in un prodotto composto

Completamento delle specifiche

Eliminazione e integrità

- L'eliminazione di un prodotto o di una SKU dal catalogo comporta la cancellazione di tutte le configurazioni dei clienti che contengono tale elemento, anche se solo parzialmente
- La rimozione di un sottoprodotto da un composto (botone RIMUOVI / link RIMUOVI nella versione HTML) non elimina il sottoprodotto: esso rimane nel catalogo come prodotto senza padre

Configurazioni

- Il prezzo totale di una configurazione viene calcolato al momento del salvataggio in base ai prezzi correnti del catalogo (price snapshotting): il prezzo salvato non varia se il catalogo viene aggiornato in seguito
- Il nome di una configurazione non può essere vuoto
- Le configurazioni nella pagina LE MIE CONFIGURAZIONI sono ordinate per data di ultima modifica decrescente

Aggiunta alle specifiche

Fascia di prezzo

- Per i prodotti semplici, prezzo_min e prezzo_max vengono calcolati automaticamente come MIN e MAX dei prezzi delle SKU associate, al momento del salvataggio del prodotto
- Per i prodotti composti, la fascia di prezzo (min/max) è inserita liberamente dal fornitore, con il vincolo che prezzo_max $\geq$ prezzo_min e che entrambi siano ≥ 0
- La fascia di prezzo del prodotto composto radice è mostrata nella pagina SCELTA SKU durante la configurazione
- Il cliente non può salvare una configurazione il cui prezzo totale è inferiore a prezzo_min o superiore a prezzo_max del prodotto composto radice; in caso di errore la pagina SCELTA SKU viene riproposta con le selezioni precedenti mantenute

Functional requirements

Versione HTML puro

- L'interfaccia è realizzata come applicazione multi-pagina; ogni interazione può comportare il ricarico completo della pagina
- Nessun codice JavaScript è utilizzato
- Il rendering delle pagine avviene lato server tramite template Thymeleaf
- In caso di errore di validazione, la pagina viene ricaricata con i valori inseriti dall'utente mantenuti nei campi e i messaggi di errore visualizzati

Versione JavaScript

- L'interfaccia è realizzata come Single Page Application (una pagina per ruolo)
- Le richieste al server avvengono in modo asincrono; il ricarico della pagina non è mai completo
- Le risposte del server sono in formato JSON; il rendering è eseguito lato client

Functional requirements

Comuni a entrambe le versioni

Validazione

- La validità dell'input è controllata sia lato client (attributi HTML5 / JavaScript) sia lato server (Servlet), prima di qualsiasi accesso al database

Autenticazione e autorizzazione

- L'autenticazione avviene tramite verifica delle credenziali nel database; la password è hashata utilizzando l'algoritmo BCrypt e non in chiaro
- Lo stato di autenticazione è mantenuto tramite sessione del Servlet container (JSESSIONID)
- Un filtro di accesso (AccessControlFilter) intercetta tutte le richieste: le richieste non autenticate verso risorse protette sono reindirizzate al login
- Un filtro di ruolo (RoleFilter) verifica che il ruolo dell'utente corrisponda all'area acceduta (/fornitore/* o /cliente/*); in caso di mismatch la richiesta è rifiutata con 403
- Ogni Servlet verifica il ruolo dell'utente in modo indipendente dal filtro

Functional requirements

Comuni a entrambe le versioni

Sicurezza

- Tutte le operazioni mutanti (POST) sono protette da CSRF tramite il pattern Synchronizer Token (token generato con SecureRandom, confronto constant-time)
- Ogni query al database usa PreparedStatement per prevenire SQL injection; l'uso di Statement senza parametrizzazione è vietato
- Le query che accedono a dati dell'utente (configurazioni) filtrano sempre per username, impedendo l'accesso a risorse di altri utenti

Transazionalità

- Le operazioni che coinvolgono più tabelle (salvataggio configurazione, eliminazione prodotto) sono eseguite in transazione con setAutoCommit(false) e rollback in caso di errore

Sommario (fornitore)

Viste e componenti

- Login
 - Form login
- Home Fornitore
 - Form crea SKU
 - Form crea prodotto semplice
 - Form crea prodotto composto
 - Visualizzazione oggetto salvato
 - Messaggio benvenuto
 - Link pagina ricerca
 - Bottone Logout

Eventi e Azioni

- Login
 - Verifica credenziali e reindirizzamento in base al ruolo
- Salva SKU
 - Inserimento SKU nel DB
- Salva prodotto semplice
 - Inserimento prodotto e associazioni SKU
- Salva prodotto composto
 - Inserimento prodotto e sottoprodotti
- Click pagina ricerca
 - Naviga a pagina di ricerca
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario (fornitore)

Viste e componenti

- Cerca Prodotti
 - Form ricerca
 - Lista risultati
 - Dettaglio oggetto trovato
- Bottone Logout
- Bottone Home

Eventi e Azioni

- Click cerca
 - Ricerca case-insensitive su catalogo
- Click elemento risultato ricerca
 - Visualizzazione dettaglio oggetto
- Click rimuovi
 - Rimozione associazione
- Click elimina
 - Eliminazione oggetto a cascata sulle configurazioni
- Click Home
 - Navigazione alla pagina Home Fornitore
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario (cliente)

Viste e componenti

- Home Cliente
 - Lista prodotti composti paginata
 - Bottoni precedenti successivi
 - Messaggio di benvenuto
 - Bottone le mie configurazioni
 - Bottone Logout

Eventi e Azioni

- Click elemento prodotto composto
 - Navigazione alla schermata Scelta SKU
- Click Le mie Configurazioni
 - Navigazione alla schermata Le mie Configurazioni
- Click Home
 - Navigazione alla pagina Home
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario (cliente)

Viste e componenti

- Scelta SKU
 - Albero dei sottoprodotti espandibile
 - Bottoni di espansione
 - Form scelta nome configurazione
 - Bottone salva configurazione
 - Bottone le mie configurazioni
 - Botton Home
 - Bottone Logout

Eventi e Azioni

- Click salva configurazione con campi corretti
 - Navigazione alla schermata Dettaglio configurazione
- Click salva configurazione con campi errati
 - Notifica errore, viene riproposta la pagina con campi precompilati
- Click Le mie Configurazioni
 - Navigazione alla pagina Le mie Configurazioni
- Click Home
 - Navigazione alla pagina Home
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario (cliente)

Viste e componenti

- Dettaglio configurazione
 - SKU selezionate con rispettivi dati e prezzo configurazione
 - Bottone le mie configurazioni
 - Botton Home
 - Bottone Logout

Eventi e Azioni

- Click Le mie Configurazioni
 - Navigazione alla pagina Le mie Configurazioni
- Click Home
 - Navigazione alla pagina Home
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario (cliente)

Viste e componenti

- Le mie Configurazioni
 - Lista configurazioni salvate dell'utente
 - Bottone Vedi Dettaglio
 - Bottone Modifica
 - Bottone Clona
 - Bottone Elimina
 - Bottone Nuova configurazione
 - Botton Home
 - Bottone Logout

Eventi e Azioni

- Click Vedi Dettaglio
 - Navigazione alla pagina Dettaglio configurazione
- Click Modifica
 - Navigazione alla pagina Scelta SKU con valori preselezionati
- Click Clona
 - Clona configurazione e ricarica la pagina
- Click Elimina
 - Elimina configurazione e ricarica la pagina
- Click Nuova configurazione
 - Navigazione alla pagina Selta SKU
- Click Home
 - Navigazione alla pagina Home
- Click Logout
 - Logout dell'utente e navigazione schermata Login

Sommario JS (fornitore)

Viste e componenti

- Login
 - Form login
- Home Fornitore JS
 - Form crea SKU
 - Attributi SKU editabili
 - Form crea prodotto di primo livello
 - Lista nidificata con attributi editabili
 - Bottoni +/-/* per prodotti e SKU
 - Messaggio benvenuto
 - Bottone Logout

Eventi e Azioni

- Login
 - Verifica credenziali e reindirizzamento
- Click Salva SKU
 - Inserimento SKU nel DB
- Click attributo SKU
 - Rende il valore editabile
- Mouseout campo attributo SKU
 - Salva attributo nel DB
- Click Salva Prodotto
 - Inserimento prodotto nel DB
- Click attributo prodotto
 - Rende valore editabile
- Click Logout
 - Logout e redirect a Login

Sommario JS (fornitore)

Viste e componenti

- Home Fornitore JS
- Menu + su prodotto composto
- Menu + su prodotto semplice
- Form sottoprodotto semplice inline
- Form nuova SKU inline
- Form selezione SKU esistente

Eventi e Azioni

- Click + su prodotto composto
 - Mostra menu (sottoprodotto composto/semplce)
- Click sottoprodotto composto
 - Aggiunge prodotto composto figlio lato client
- Click sottoprodotto semplice
 - Aggiunge prodotto semplice figlio lato client
- Click + su prodotto semplice
 - Mostra menu (Nuova SKU/SKU esistente)
- Click Nuova SKU
 - Apre form nuova SKU
- Click SKU esistente
 - Apre form selezione SKU a scelta multipla
- Click - su prodotto (se ha padre)
 - Rimuove relazione padre-figlio lato client
- Click -* su prodotto
 - Elimina prodotto e sottoprodotti lato client
- Click - su SKU
 - Rimuove associazione SKU-prodotto lato client
- Click -* su SKU
 - Elimina SKU lato client

Sommario JS (fornitore)

Viste e componenti

- Home Fornitore JS
- Sezione ricerca integrata
- Form ricerca
- Lista risultati con dettaglio editabile
- Bottoni Undo/Redo

Eventi e Azioni

- Click Cerca
 - Ricerca case-insensitive, aggiornamento parziale vista
- Click risultato ricerca
 - Visualizzazione dettaglio con attributi editabili
- Mouseout campo in risultato ricerca
 - Salva attributo nel DB
- Click Rimuovi associazione (in risultato)
 - Rimozione associazione, aggiornamento parziale vista
- Click Elimina (in risultato)
 - Eliminazione a cascata, aggiornamento parziale vista
- Click Undo
 - Annulla ultima operazione lato client
- Click Redo
 - Ripristina operazione annullata lato client
- Click Logout
 - Logout e redirect a Login

Sommario JS (cliente)

Viste e componenti

- Home Cliente JS
- Lista prodotti composti paginata
- Bottoni Precedenti/Successivi
- Vista configurazione: albero espandibile
- Bottoni espansione nodo
- Menu a tendina SKU per prodotti semplici
- Form nome configurazione
- Bottone Salva configurazione
- Vista dettaglio configurazione
- Bottone Le mie Configurazioni
- Bottone Logout

Eventi e Azioni

- Login
- Verifica credenziali e reindirizzamento
- Caricamento Home
- Fetch catalogo, aggiornamento parziale vista
- Click Precedenti/Successivi
- Aggiornamento paginazione (aggiornamento parziale vista)
- Click prodotto composto
- Fetch albero prodotto, aggiornamento parziale vista
- Click espandi nodo
- Espande figli del nodo (dati in RAM, no fetch)
- Click Salva configurazione (corretta)
- Price snapshotting, salvataggio nel DB, vista dettaglio
- Click Salva configurazione (errata)
- Notifica errore, ripropone form con selezioni mantenute
- Click Logout
- Logout e redirect a Login

Sommario JS (cliente)

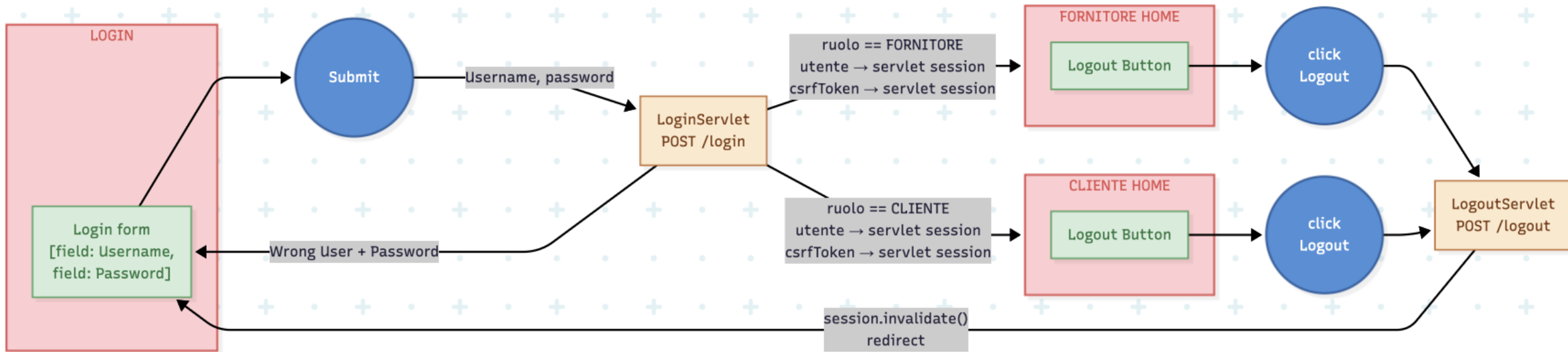
Viste e componenti

- Home Cliente JS
- Sezione Le mie Configurazioni
- Lista configurazioni (per data modifica desc.)
- Bottone Dettaglio
- Bottone Modifica
- Bottone Clona
- Bottone Elimina
- Vista dettaglio configurazione
 - SKU selezionate con dati e prezzo congelato
- Bottone Nuova Configurazione
- Bottone Home
- Bottone Logout

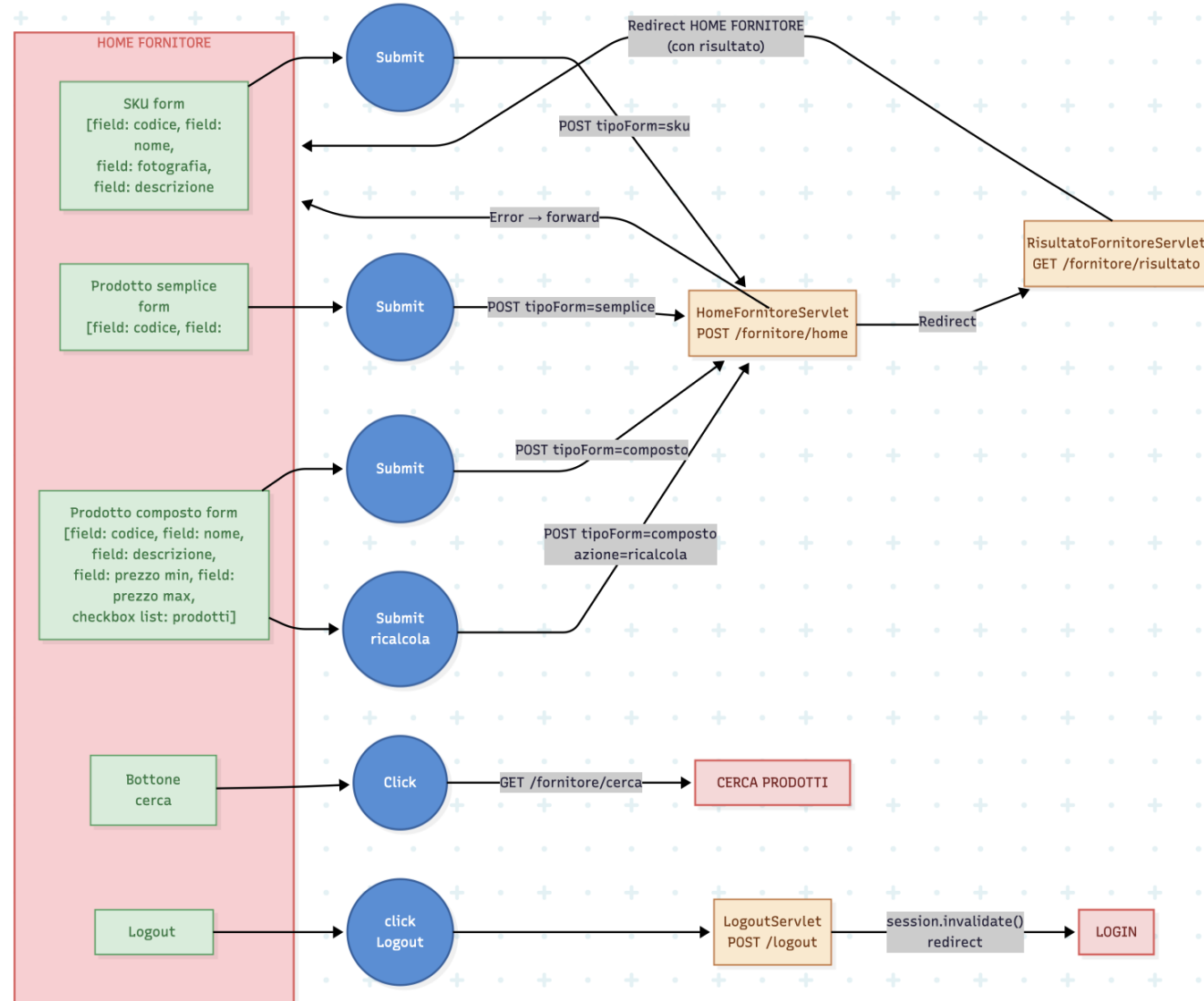
Eventi e Azioni

- Click Le mie Configurazioni
 - Fetch configurazioni, aggiornamento parziale vista
- Click Dettaglio
 - Fetch dettaglio con prezzi congelati, aggiornamento vista
- Click Modifica
 - Fetch configurazione + albero, vista form precompilato
- Click Salva modifiche
 - Aggiornamento configurazione nel DB, vista dettaglio
- Click Clona
 - Inserimento copia nel DB, aggiornamento elenco
- Click Elimina
 - Eliminazione nel DB, aggiornamento elenco
- Click Nuova Configurazione
 - Aggiornamento vista catalogo per scelta prodotto
- Click Home
 - Aggiornamento vista catalogo
- Click Logout
 - Logout e redirect a Login

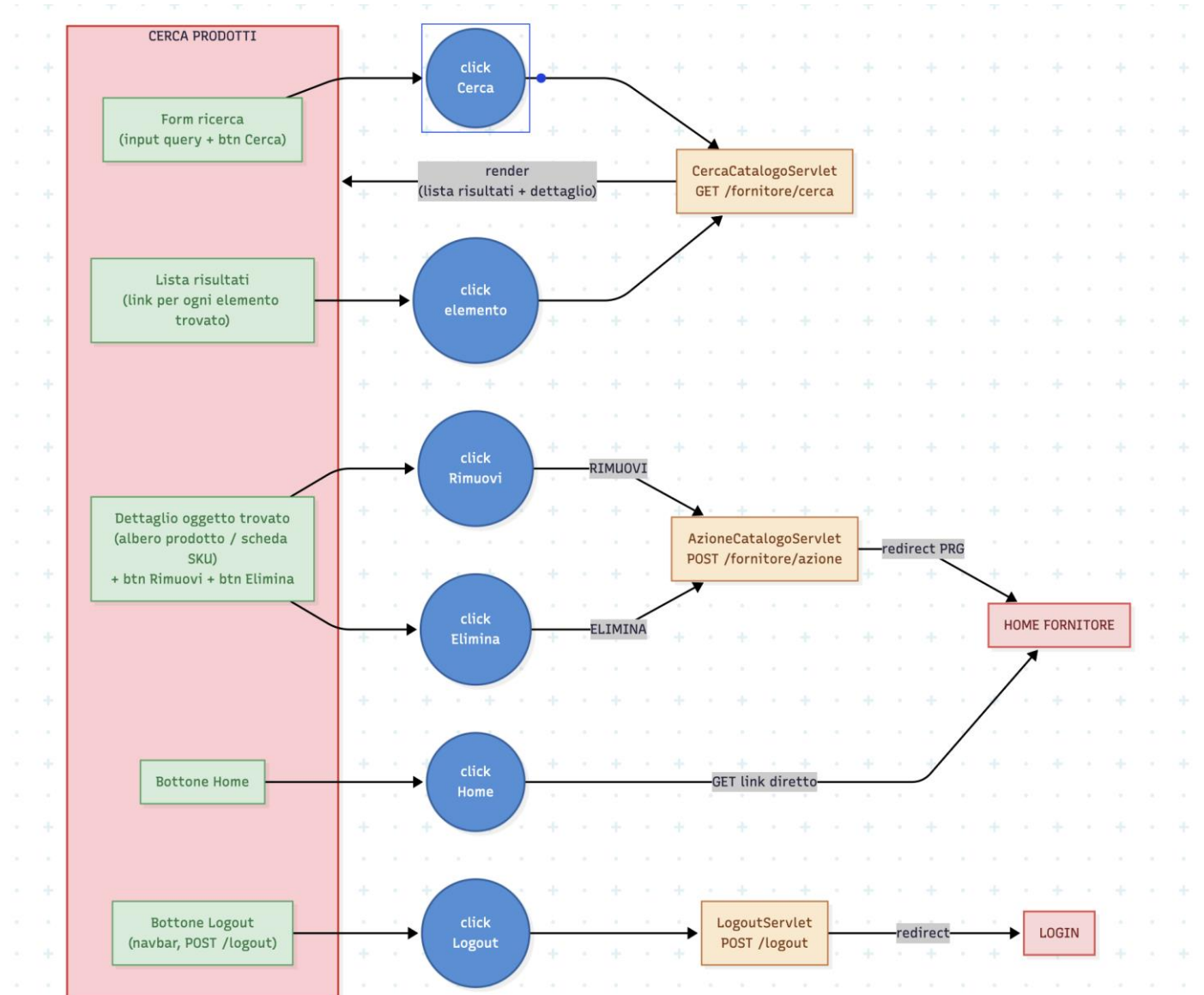
Application design (login)



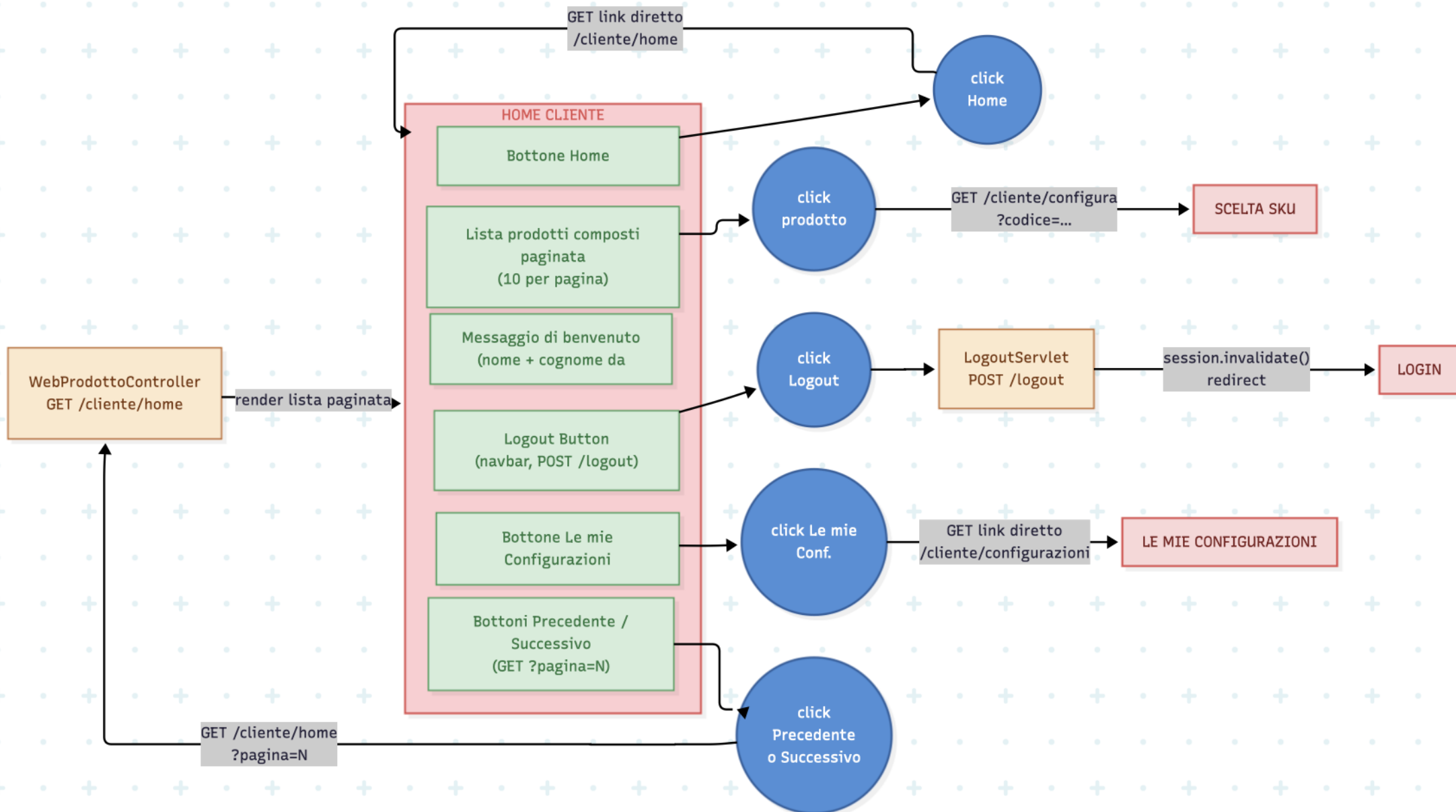
Application design (home fornitore)



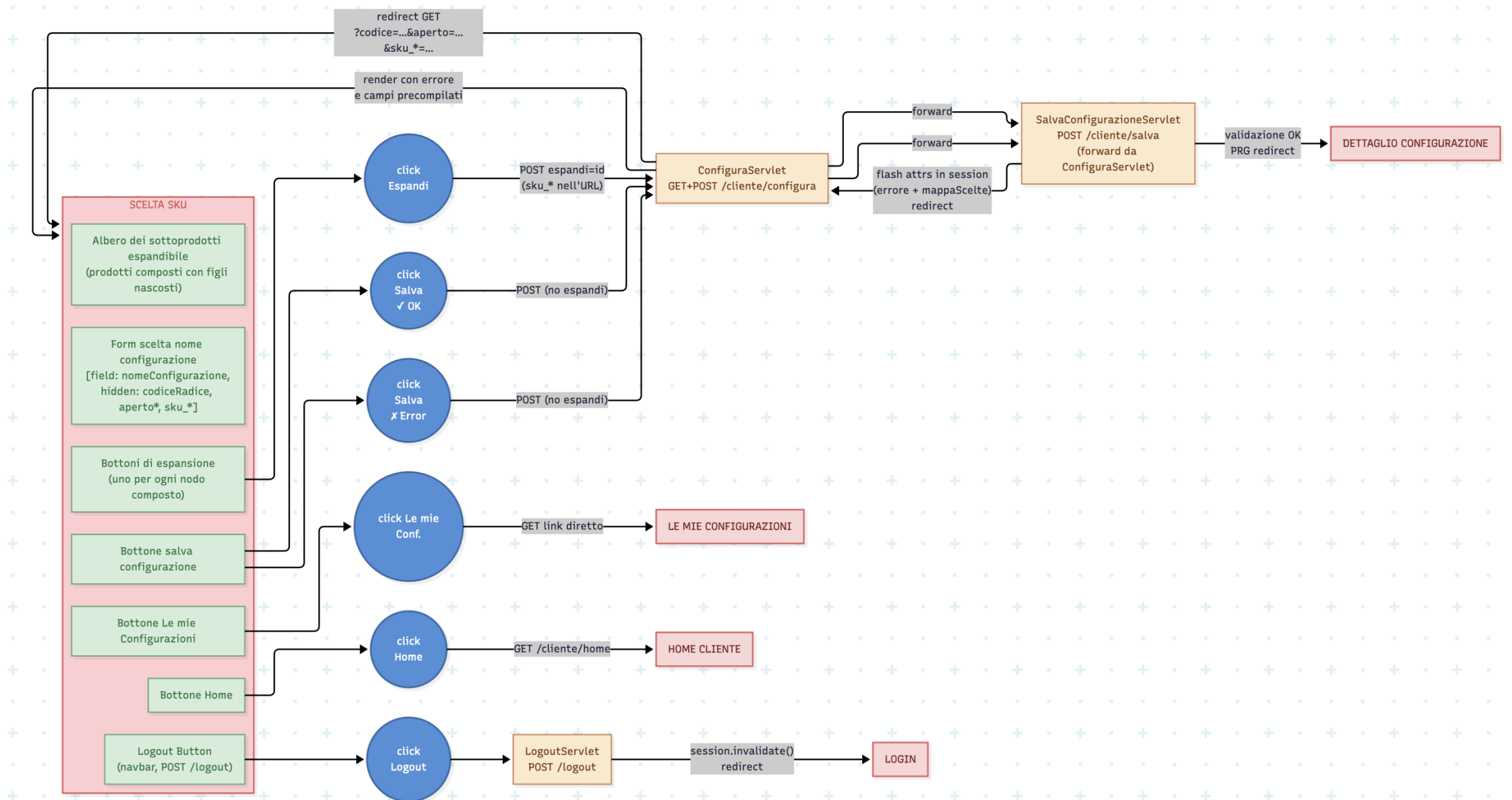
Application design (cerca prodotti)



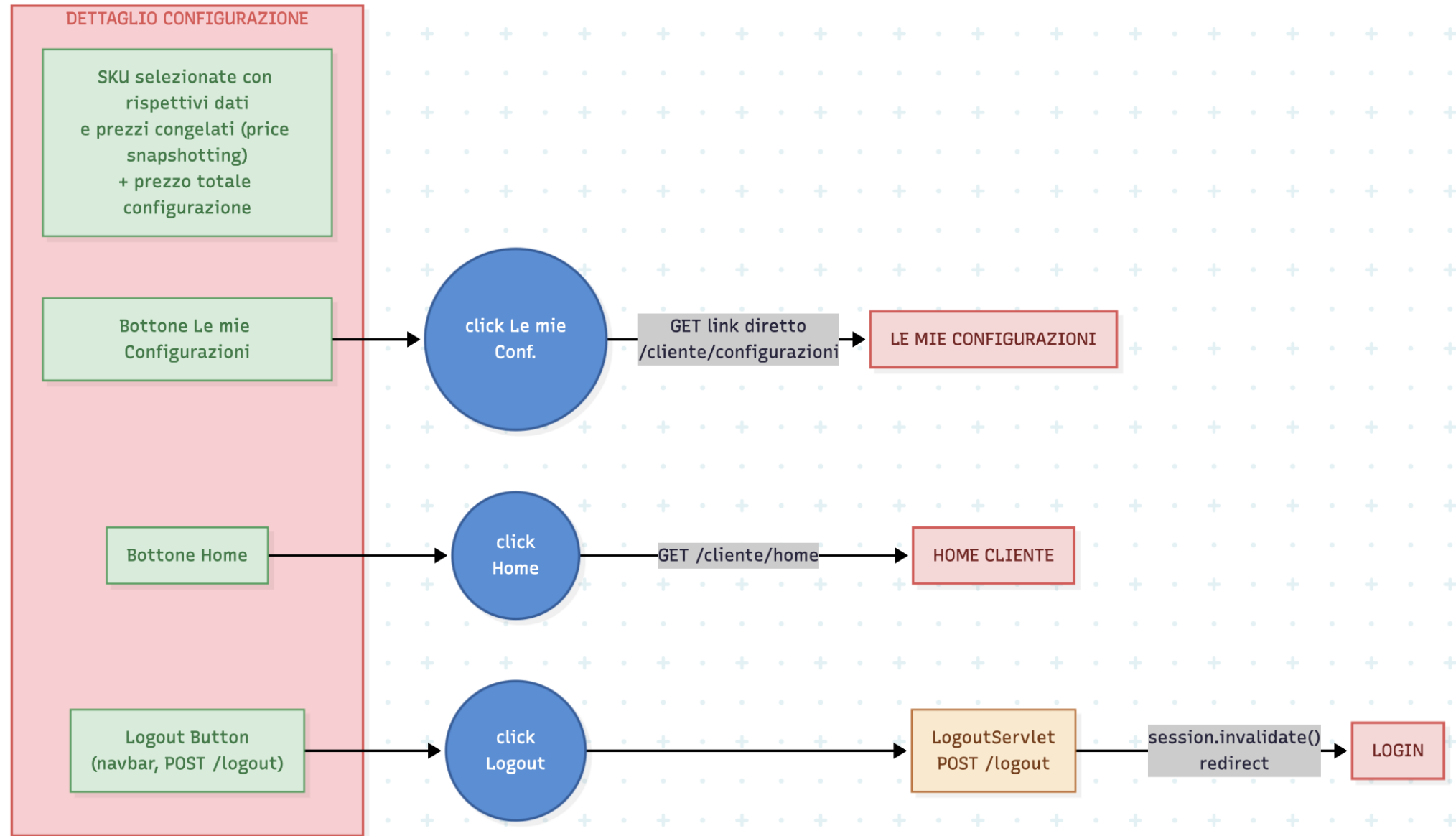
Application design (home cliente)



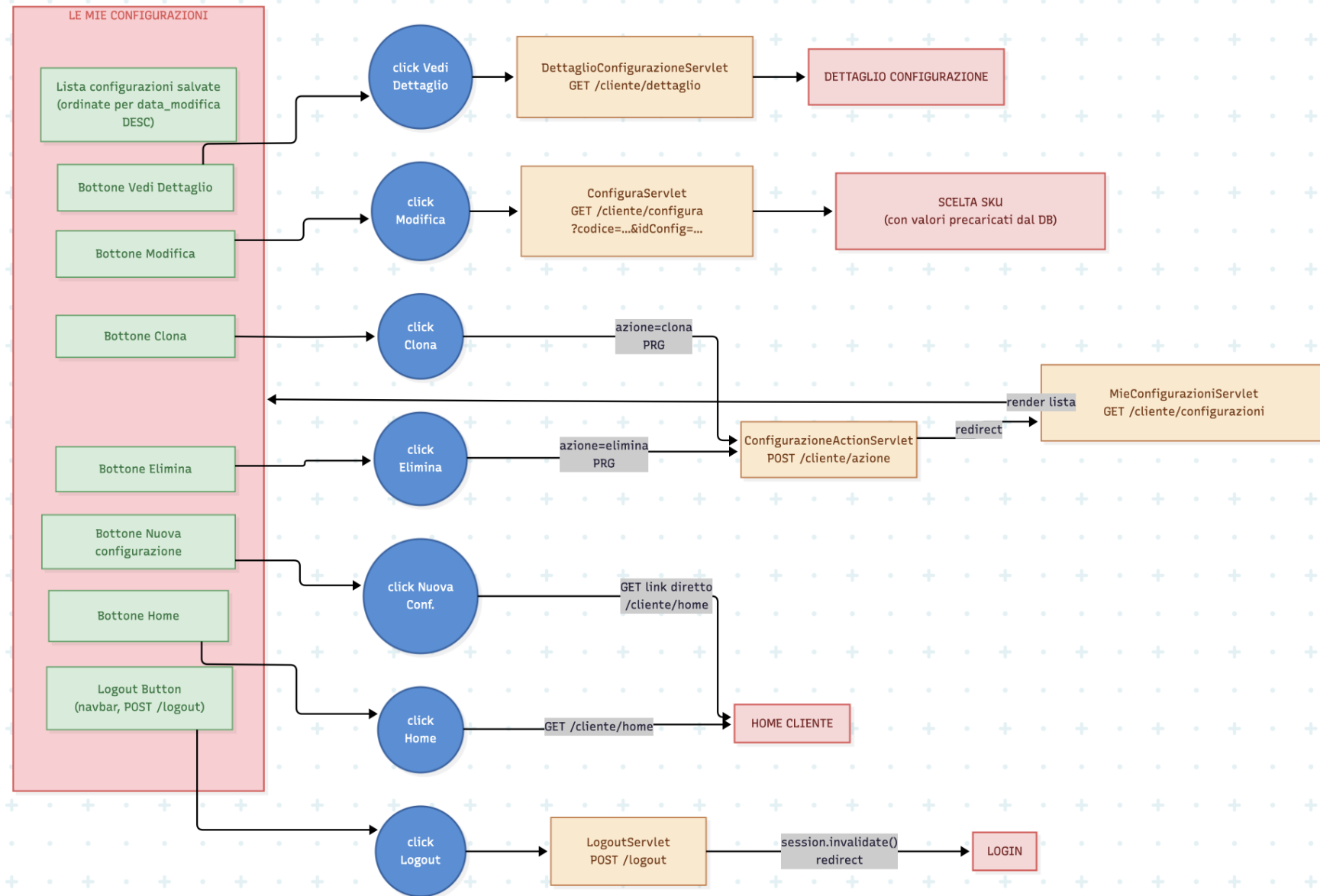
Application design (Scelta SKU cliente)



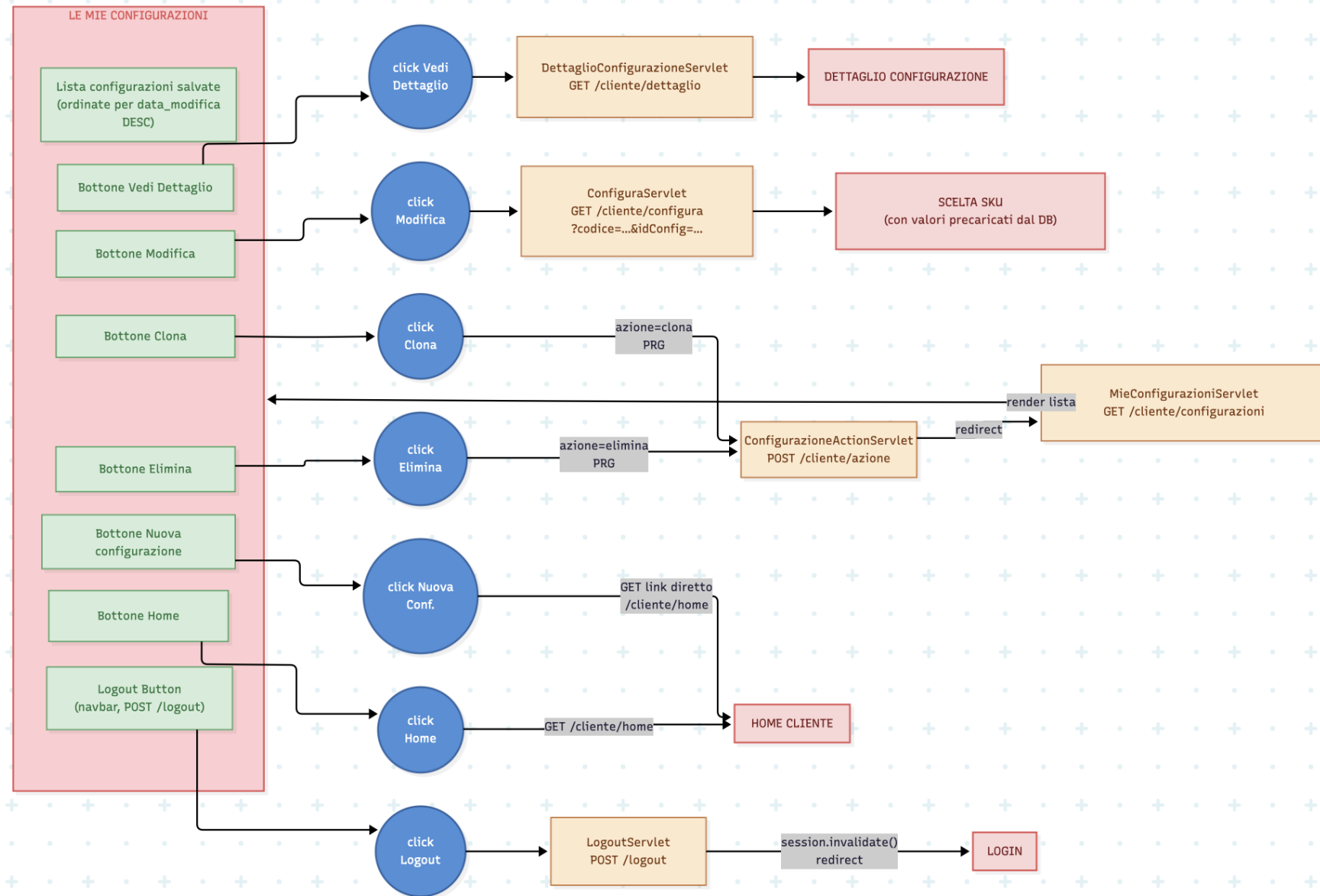
Application design (Scelta SKU cliente)



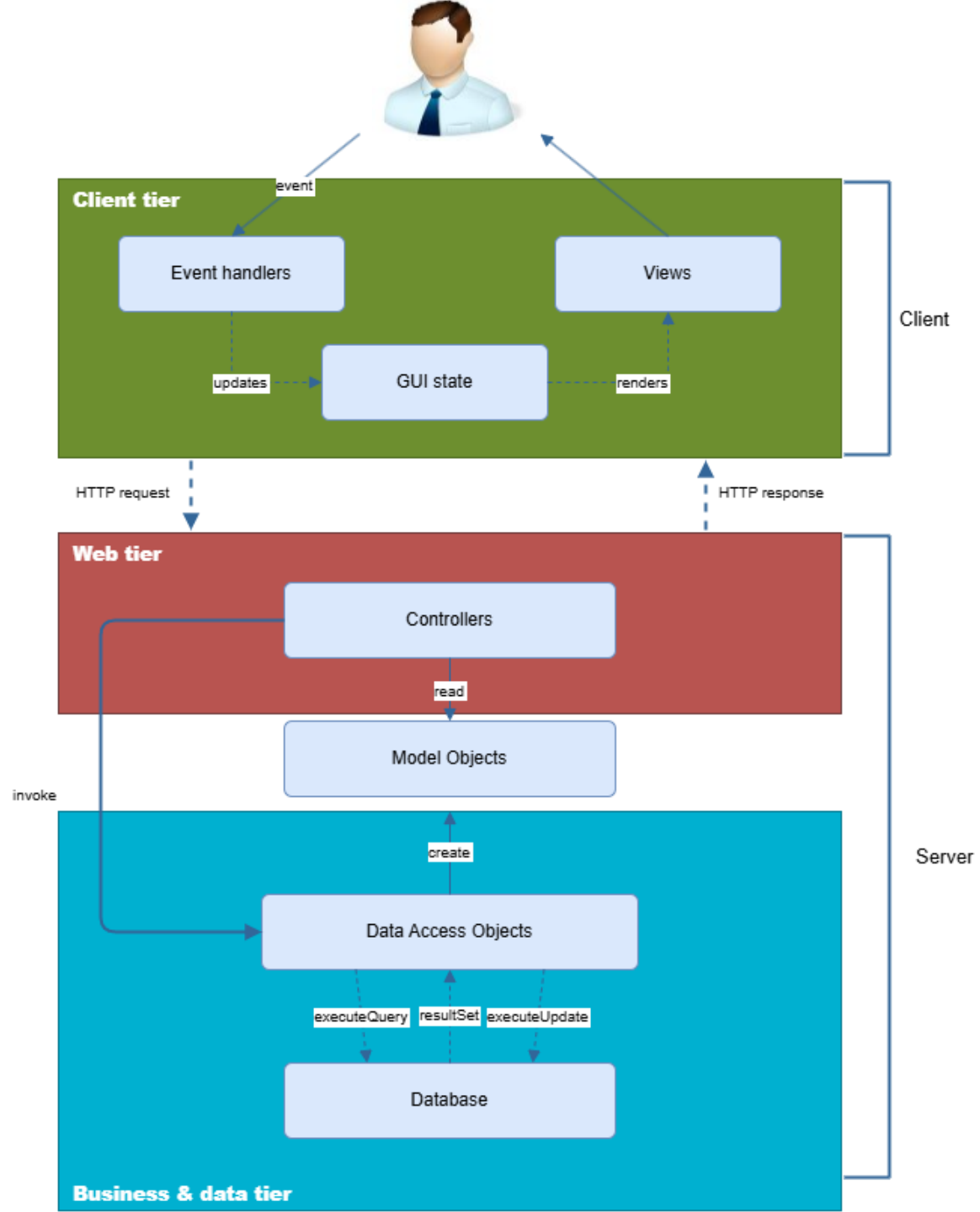
Application design (Scelta SKU cliente)



Application design (Scelta SKU cliente)



Architettura



Modulo SPA

Controller / Event Handler

SPA · Fornitore

makeCall indica una funzione che fa una chiamata asincrona al server

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	api.fetchJson (POST)	POST username, password	LoginServlet (servlet)
Home page → load	AppFornitore.init (makeCall)	GET (nessun parametro)	HomeFornitoreServlet (servlet)
Form SKU → submit	AppFornitore.handleSubmitSku (makeCall)	POST dati SKU + foto	ApiSkuController (servlet)
Form prodotto semplice → submit	AppFornitore.handleSubmitSempl ice (makeCall)	POST JSON (codice, nome, SKU)	ApiProdottoTreeController (servlet)
Form prodotto composto → submit	AppFornitore.handleCreaCompos to (makeCall)	POST JSON (codice, nome, prezzi, figli)	ApiProdottoTreeController (servlet)
Tree editor → click Aggiungi SKU	AppFornitore.handleTreeAddSku	—	—
Tree editor → click Scollega / Elimina nodo	AppFornitore.handleTreeUnlink / handleTreeDelete	—	—
Tree editor → focusout campo editabile	AppFornitore.handleTreeInlineEdit	—	—
Tree editor → click Salva	AppFornitore.handleSalvaTree (makeCall)	POST JSON (azioni pendenti)	ApiSyncController (servlet)
Ricerca → form submit	AppFornitore.handleSearch (makeCall)	GET ?q=...	ApiRicercaController (servlet)
Risultati → click su item	AppFornitore.handleExpandSearc hResult (makeCall)	GET ?id=...	ApiProdottoController (servlet)
SKU → click Elimina	AppFornitore.handleEliminaSku (makeCall)	DELETE ?id=...	ApiSkuController (servlet)
Logout	redirect window.location.href	GET /logout	LoginServlet (servlet)

Controller / Event Handler

makeCall indica una funzione che fa una chiamata asincrona al server

SPA · Cliente

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	api.fetchJson (POST)	POST username, password	LoginServlet (servlet)
Home page → load	AppCliente.init → caricaCatalogo (makeCall)	GET (nessun parametro)	HomeClienteServlet (servlet)
Catalogo → click prodotto	AppCliente.apriConfigurazione (makeCall)	GET ?codice=...	ApiConfigurazioneController (servlet)
Configura → form submit (nuova)	AppCliente.handleSalvaConfigurazione (makeCall)	POST JSON (nome, codiceRadice, scelte)	ApiConfigurazioneController (servlet)
Configura → form submit (modifica)	AppCliente.handleSalvaConfigurazione (makeCall)	PUT JSON (nome, scelte)	ApiConfigurazioneController (servlet)
Le mie configurazioni → load	AppCliente.caricaConfigurazioni (makeCall)	GET /api/cliente/configurazioni	ApiConfigurazioneController (servlet)
Configurazione → click Dettaglio	AppCliente.apriDettaglio (makeCall)	GET /api/cliente/configurazioni/{id}	ApiConfigurazioneController (servlet)
Configurazione → click Modifica	AppCliente.apriModifica (makeCall)	GET /api/cliente/configurazioni/{id}	ApiConfigurazioneController (servlet)
Configurazione → click Clona	AppCliente.cloneConfigurazione (makeCall)	POST JSON (payload clonato)	ApiConfigurazioneController (servlet)
Configurazione → click Elimina	AppCliente.eliminaConfigurazione (makeCall)	DELETE /api/cliente/configurazioni/{id}	ApiConfigurazioneController (servlet)
Logout	redirect window.location.href	GET /logout	LoginServlet (servlet)

Eventi & Azioni

SPA · Fornitore

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione
Home page → load	Aggiorna view con SKU e prodotti orfani	GET (nessun parametro)	Estrazione SKU e prodotti orfani
Form SKU → submit	Controllo dati; invio multipart	POST dati SKU + foto	Controllo codice univoco; upload foto; inserimento SKU
Form prodotto semplice → submit	Controllo dati; invio JSON	POST JSON (codice, nome, SKU)	Controllo codice univoco; inserimento prodotto + associazioni SKU
Form prodotto composto → submit	Controllo dati e vincoli prezzo; invio JSON	POST JSON (codice, nome, prezzi, figli)	Verifica vincoli prezzo e profondità ≤ 3; inserimento prodotto + figli
Tree editor → Aggiungi figlio / SKU	Aggiunta nodo/SKU al DOM; accodamento azione	—	—
Tree editor → click Salva	Invio batch azioni pendenti	POST JSON (azioni pendenti)	Esecuzione transazionale delle azioni; aggiornamento DB
Ricerca → form submit	Aggiorna view con risultati	GET ?q=...	Ricerca full-text su SKU e prodotti
Risultati → click su item	Aggiorna view con albero / dettaglio SKU	GET ?id=...	Estrazione albero prodotto o dati SKU
SKU → click Elimina	Rimozione dalla view	DELETE ?id=...	Eliminazione fisica SKU e configurazioni associate
Logout	Cancellazione stato locale; redirect login	GET /logout	Invalidazione sessione

Eventi & Azioni

SPA · Cliente

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione
Home page → load	Aggiorna view con catalogo prodotti (Z-A, paginato)	GET (nessun parametro)	Estrazione prodotti composti radice
Catalogo → click prodotto	Aggiorna view con albero configurazione	GET ?codice=...	Estrazione albero prodotto
Configura → form submit (nuova)	Controllo dati; aggiorna view con dettaglio	POST JSON (nome, codiceRadice, scelte SKU)	Controllo dati; price snapshotting; inserimento configurazione
Configura → form submit (modifica)	Controllo dati; aggiorna view con dettaglio	PUT JSON (nome, scelte SKU)	Controllo dati; price snapshotting; aggiornamento configurazione
Le mie configurazioni → load	Aggiorna view con elenco configurazioni	GET /api/cliente/configurazioni	Estrazione configurazioni del cliente
Configurazione → click Dettaglio	Aggiorna view con dettaglio e prezzi congelati	GET /api/cliente/configurazioni/{id}	Estrazione dettaglio + prezzi congelati
Configurazione → click Modifica	Aggiorna view con form pre-compilato	GET /api/cliente/configurazioni/{id}	Estrazione configurazione + albero + scelte
Configurazione → click Clona	Aggiunge nuova voce all'elenco	POST JSON (payload clonato)	Inserimento copia configurazione
Configurazione → click Elimina	Rimozione dalla view	DELETE /api/cliente/configurazioni/{id}	Eliminazione configurazione
Logout	Cancellazione stato locale; redirect login	GET /logout	Invalidazione sessione

Modulo SSR

Controller / Event Handler

makeCall indica una funzione che fa una chiamata asincrona al server

SSR · Fornitore

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	<form action="/login">	POST username, password	LoginServlet (servlet)
Home page → load		GET (nessun parametro)	HomeFornitoreServlet (servlet)
Form SKU → submit	<form> tipoForm=sku	POST (dati SKU + foto)	HomeFornitoreServlet.handleCreaSku
Form prodotto semplice → submit	<form> tipoForm=semplice	POST (codice, nome, IDs SKU)	HomeFornitoreServlet.handleCreaSemplice
Form prodotto composto → Ricalcola	<form> azione=ricalcola	POST azione=ricalcola	HomeFornitoreServlet.handleRicalcolaComposto
Form prodotto composto → submit	<form> tipoForm=composto	POST (dati composto + IDs figli)	HomeFornitoreServlet.handleCreaComposto
Risultato → click Elimina	<form action="/fornitore/azione"> azione=ELIMINA	POST (tipo, ID oggetto)	AzioneCatalogoServlet (servlet)
Risultato → click Rimuovi	<form action="/fornitore/azione"> azione=RIMUOVI	POST (tipo, ID oggetto, ID padre)	AzioneCatalogoServlet (servlet)
Ricerca → form submit	<form action="/fornitore/cerca">	GET ?keyword=...	CercaCatalogoServlet (servlet)
Risultato ricerca → load	redirect interno	GET (nessun parametro)	RisultatoFornitoreServlet (servlet)
Logout	<form action="/logout">	POST	LogoutServlet (servlet)

Controller / Event Handler

makeCall indica una funzione che fa una chiamata asincrona al server

SSR · Cliente

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	<form action="/login">	POST username, password	LoginServlet (servlet)
Home page (catalogo) → load		GET (nessun parametro)	HomeClienteServlet (servlet)
Catalogo → click prodotto		GET ?codice=...	ConfiguraServlet (servlet)
Configura → click Espandi nodo	<form> espandi=<id>	POST espandi=<id>	ConfiguraServlet.doPost → redirect GET
Configura → form submit	<form action="/cliente/configura">	POST (nome, codiceRadice, sku_*)	ConfiguraServlet → SalvaConfigurazioneServlet
Le mie configurazioni → load		GET (nessun parametro)	MieConfigurazioniServlet (servlet)
Dettaglio configurazione → load		GET ?idConfig=...	DettaglioConfigurazioneServlet (servlet)
Logout	<form action="/logout">	POST	LogoutServlet (servlet)

Eventi & Azioni

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

SSR - Fornitore

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione; redirect home
Home page → load	Aggiorna view con SKU e prodotti orfani	GET (nessun parametro)	Estrazione SKU e prodotti orfani
Form SKU → submit	Validazione dati form	POST (dati SKU + foto)	Controllo codice univoco; upload foto; inserimento SKU
Form prodotto semplice → submit	Validazione dati form	POST (codice, nome, IDs SKU)	Calcolo prezzoMin/Max da SKU; inserimento prodotto + associazioni
Form prodotto composto → Ricalcola	Nessuna azione (semplice submit)	POST azione=ricalcola	Calcolo somma prezzoMin/Max figli; salvataggio in sessione
Form prodotto composto → submit	Validazione dati form	POST (dati composto + IDs figli)	Verifica vincoli prezzo e profondità ≤ 3; inserimento transazionale
Risultato → click Elimina	Nessuna azione client	POST azione=ELIMINA (tipo, ID oggetto)	Rimozione configurazioni clienti collegate; eliminazione fisica
Risultato → click Rimuovi	Nessuna azione client	POST azione=RIMUOVI (tipo, ID, padre)	Rimozione associazione SKU/figlio; ricalcolo prezzi padre
Ricerca → form submit	Nessuna azione client	GET ?keyword=...	Ricerca testuale su SKU e prodotti
Logout	Nessuna azione client	POST	Invalidazione sessione; redirect login

Eventi & Azioni

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

SSR - Cliente

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione; redirect home
Home page (catalogo) → load	Aggiorna view con prodotti radice	GET (nessun parametro)	Estrazione prodotti composti radice
Catalogo → click prodotto	Nessuna azione client (link)	GET ?codice=...	Estrazione albero prodotto; rendering form configurazione
Configura → click Espandi nodo	Invio scelte SKU correnti come hidden field	POST espandi=<id> + sku_* + aperto=*	Redirect GET con query string aggiornata (PRG stateless)
Configura → form submit	Validazione nome non vuoto; completezza scelte SKU	POST (nome, codiceRadice, sku_*)	Controllo completezza; price snapshotting; inserimento/aggiornamento configurazione
Le mie configurazioni → load	Aggiorna view con elenco configurazioni	GET (nessun parametro)	Estrazione configurazioni del cliente (ordinate per data)
Dettaglio configurazione → load	Aggiorna view con dettaglio e prezzi congelati	GET ?idConfig=...	Estrazione dettaglio configurazione + prezzi congelati
Logout	Nessuna azione client	POST	Invalidazione sessione; redirect login

Parti comuni Cliente & Fornitore

Controller / Event Handler — Parti comuni

makeCall indica una funzione che fa una chiamata asincrona al server

SPA · Comune a Fornitore e Cliente

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	api.fetchJson (POST)	POST username, password	LoginServlet (servlet)
Home page → load (info utente + CSRF token)	AppX.init → api.getUser (makeCall)	GET /api/me	ApiUserController (servlet)
Sessione scaduta (401 su qualsiasi fetch)	api.fetchJson → redirect automatico	—	—
Logout	redirect window.location.href = 'login.html'	GET /logout	LoginServlet (servlet)

Eventi & Azioni — Parti comuni

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

SPA · Comune a Fornitore e Cliente

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione; generazione CSRF token
Home page → load (info utente + CSRF token)	Visualizza nome utente in navbar; imposta CSRF token nel meta tag	GET /api/me	Lettura sessione; restituzione username, nome, cognome, ruolo, CSRF token
Sessione scaduta (401 su qualsiasi chiamata)	Redirect automatico a login.html	—	—
Logout	Cancellazione stato locale; redirect a login.html	GET /logout	Invalidazione sessione HTTP

Controller / Event Handler — Parti comuni

makeCall indica una funzione che fa una chiamata asincrona al server

SSR · Comune a Fornitore e Cliente

Client side		Server side	
Evento	Controllore	Evento	Controllore
index → login form → submit	<form action="/login" method="POST">	POST username, password	LoginServlet (servlet)
Sessione assente / scaduta (qualsiasi pagina)	redirect automatico (nessun controller client)	— (filtri di sessione nelle servlet)	redirect a /login
Logout	<form action="/logout" method="POST">	POST	LogoutServlet (servlet)

Eventi & Azioni — Parti comuni

I controlli di validità (client e server side) e di autorizzazione (server side) all'accesso sono previsti per tutti gli eventi che li richiedono e non sono riportati per brevità

SSR · Comune a Fornitore e Cliente

Client side		Server side	
Evento	Azione	Evento	Azione
index → login form → submit	Controllo credenziali	POST username, password	Controllo credenziali; creazione sessione; CSRF token; redirect home per ruolo
Sessione assente / scaduta (qualsiasi pagina)	Nessuna azione client	—	Redirect a /login
Logout	Nessuna azione client	POST	Invalidazione sessione HTTP; redirect a /login

Server side: DAO & model objects

Controllers

- LoginServlet
- ApiUserController
- ApiSkuController
- ApiProdottoTreeController
- ApiSyncController
- ApiConfigurazioneController
- ApiRicercaController
- ApiProdottoController

■ Model objects (Beans)

- Utente
- SKU
- Prodotto
- Configurazione

■ Data Access Objects (Classes)

- UtenteDAO
 - checkCredentials(username, pwd)
- SKUDAO
 - inserisciSku(...)
 - getPrezzoReale(id)
 - eliminaDefinitivamente(...)
- ProdottoDAO
 - inserisciProdottoComposto(...)
 - getAlberoProdottoByCodice(...)
- ConfigurazioneDAO
 - inserisciTestata(...)
 - inserisciDettagliBatch(...)

Client side: view & view component

A differenza della progettazione con HTML puro, le viste vengono calcolate sul lato client

Un componente client-side della vista:

- mantiene eventuali dati del view model
- espone le funzioni che gestiscono gli eventi associati al componente e la funzione di utilità per associare gli eventi ai rispettivi gestori
- espone le funzioni che invocano i servizi del server in maniera asincrona ed eseguono il rendering dei dati provenienti dal server

- **Index**

- Login form

- Gestione del submit e degli errori

- **AppFornitore**

- init: carica dati iniziali dal server (orfani, catalogo)
 - handleSalvaTree: invia le azioni dell'editor albero al server
 - handleSubmitSku: invia i dati del nuovo componente SKU

- **AppCliente**

- caricaCatalogo: richiede i prodotti al server e aggiorna la vista
 - apriConfigurazione: renderizza l'albero per la scelta SKU
 - handleSalvaConfigurazione: invia i dati della configurazione al server

Server side: Controllers

SPA

- API Controllers (SPA)
 - ApiConfigurazioneController
 - ApiProdottoController
 - ApiProdottoTreeController
 - ApiRicercaController
 - ApiSkuController
 - ApiSyncController
 - ApiUserController

SSR

- Servlets (SSR)
 - AzioneCatalogoServlet
 - CercaCatalogoServlet
 - ConfiguraServlet
 - ConfigurazioneActionServlet
 - DettaglioConfigurazioneServlet
 - HomeClienteServlet
 - HomeFornitoreServlet
 - LoginServlet
 - LogoutServlet
 - MieConfigurazioniServlet
 - RisultatoFornitoreServlet
 - SalvaConfigurazioneServlet

Server side: DAO & Model objects

Model Objects

- Beans
 - Configurazione
 - ElementoCatalogo
 - Prodotto
 - ProdottoComposto
 - ProdottoSemplice
 - SKU
 - Utente
- DTOs
 - DettaglioDTO
 - UtenteSessionDTO
 - VoceConfigurazioneDTO
- UtenteDAO
 - checkCredentials(username, password)
- ConfigurazioneDAO
 - inserisciTestata(conf)
 - inserisciDettagliBatch(id, dettagli)
 - eliminaConfigurazione(id, user)
 - getConfigurazioneById(id, user)
 - getConfigurazioniByUtente(user)
 - updateTestata(conf)
 - eliminaConfigurazioniPerComponente(id, tipo)

Data Access Objects

- SKUDAO
 - findAll(), findById(id)
 - findByCodice(codice)
 - insert(sku), update(sku)
 - getPrezzoReale(id)
 - search(query)
 - eliminaDefinitivamente(id)
- ProdottoDAO
 - getProdottiRadice()
 - findAll(), findById(id), findByCodice(cod)
 - getAlberoProdotto(id)
 - insertSemplice(...), insertComposto(...)
 - addSku(idProdotto, idSku)
 - addFiglio(idPadre, idFiglio)
 - search(query)
 - calcolaPrezziDaSku(id), ricalcolaPrezziComposto(id)
 - rimuoviAssociazioneSku(idProd, idSku)
 - rimuoviFiglio(idFiglio)
 - eliminaDefinitivamente(id)

Client side: view & view component (1/2)

Introduzione

A differenza della progettazione con HTML puro, le viste vengono calcolate sul lato client.

Un componente client-side della vista:

- mantiene eventuali dati del view model
- espone le funzioni che gestiscono gli eventi associati al componente
- espone le funzioni che invocano i servizi del server in maniera asincrona
- esegue il rendering dei dati provenienti dal server

Index / Login form

- Submit form (login.js) con fetch e redirect

Componente Cliente

AppCliente

- Inizializzazione & Navigazione
 - init, bindGlobalEvents, switchSection
- Caricamento & Rendering Liste
 - caricaCatalogo, renderPaginaCatalogo
 - caricaConfigurazioni
- Gestione Configurazione
 - apriConfigurazione, apriModifica, apriDettaglio
 - buildNodoConfigura, buildNodoModifica, buildNodoDettaglio
 - espandiNodo, raccogliScelte
- Azioni Server (Async)
 - handleSalvaConfigurazione
 - cloneConfigurazione
 - eliminaConfigurazione
- Utilità
 - mostraMessaggio, confermaAzione

Client side: view & view component (2/2)

AppFornitore

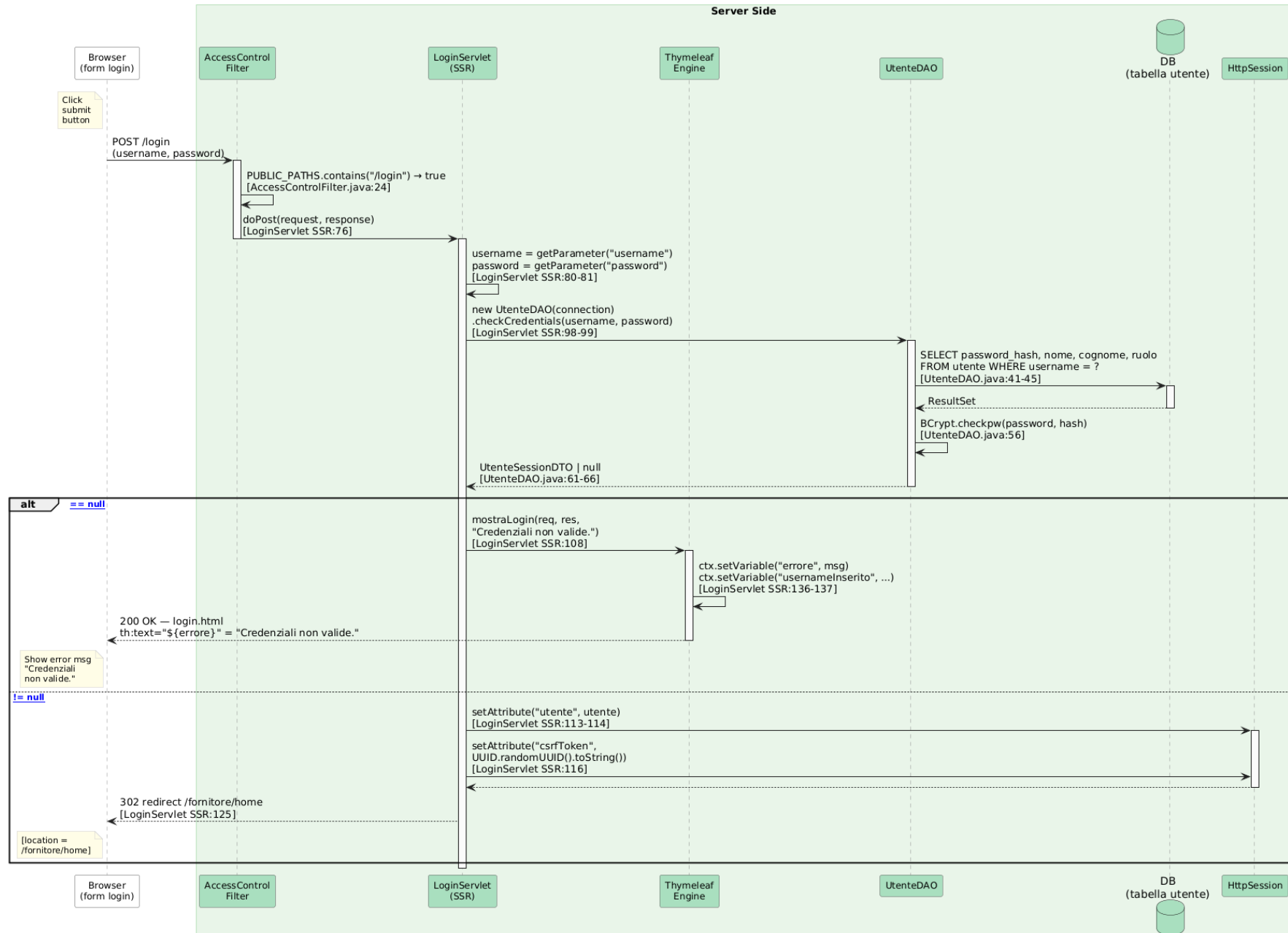
- Inizializzazione & Navigazione
 - init, bindGlobalEvents, switchSection
- Creazione Componenti
 - handleSubmitSemplice, handleCreaComposto
 - handleSubmitSku, renderSkuCreato
- Ricerca & Liste
 - handleSearch, handleExpandSearchResult
 - renderSkuCheckboxList, renderOrfaniCheckboxList
- Gestione Prezzi
 - ricalcolaRiepilogoPrezzi, handleRicalcolaPrezzo

AppFornitore (Cont.)

- Tree Editor: Rendering & Costruzione
 - renderTreeView, buildNodeUI, buildSkuUI
 - raccogliSkuDaAlbero, getNodeLevel
- Tree Editor: Code di Azioni Locali
 - enqueueAction (aggiunge l'azione in RAM)
 - handleTreeAddChild, submitTreeAddChild
 - handleTreeAddSku, submitTreeAddSku
 - handleTreeUnlink, handleTreeUnlinkSku
 - handleTreeDelete, handleTreeDeleteSku
 - handleTreeInlineEdit, handleSkuInlineEdit
- Azioni Server (Async)
 - handleSalvaTree (invia il batch di azioni pendenti)
 - handleEliminaSku, deleteProdottoFromDB
- Utilità
 - mostraMessaggio, confermaAzione

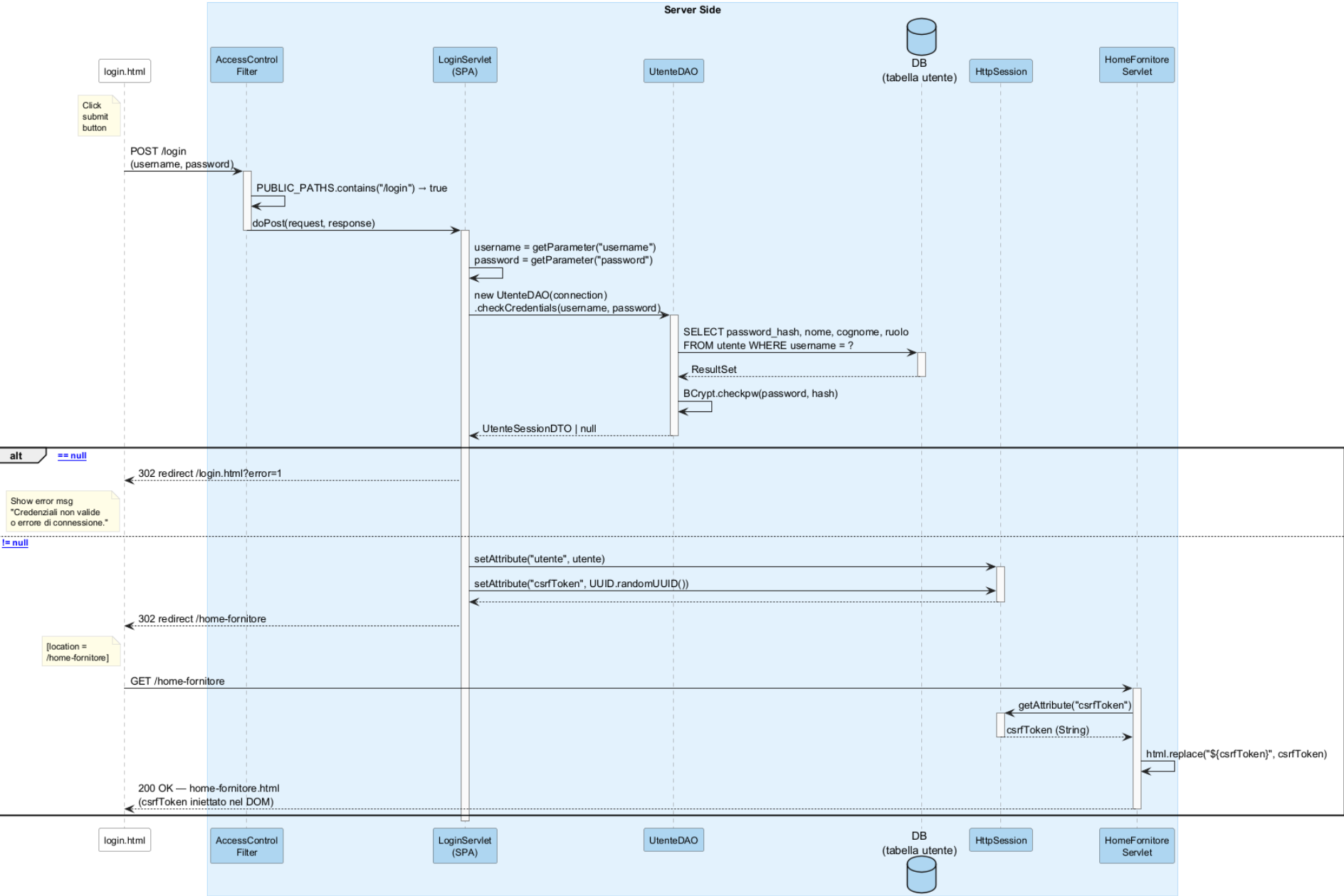
Evento: Login (SSR — Thymeleaf)

Colore	Lato
	Client side
	Server side



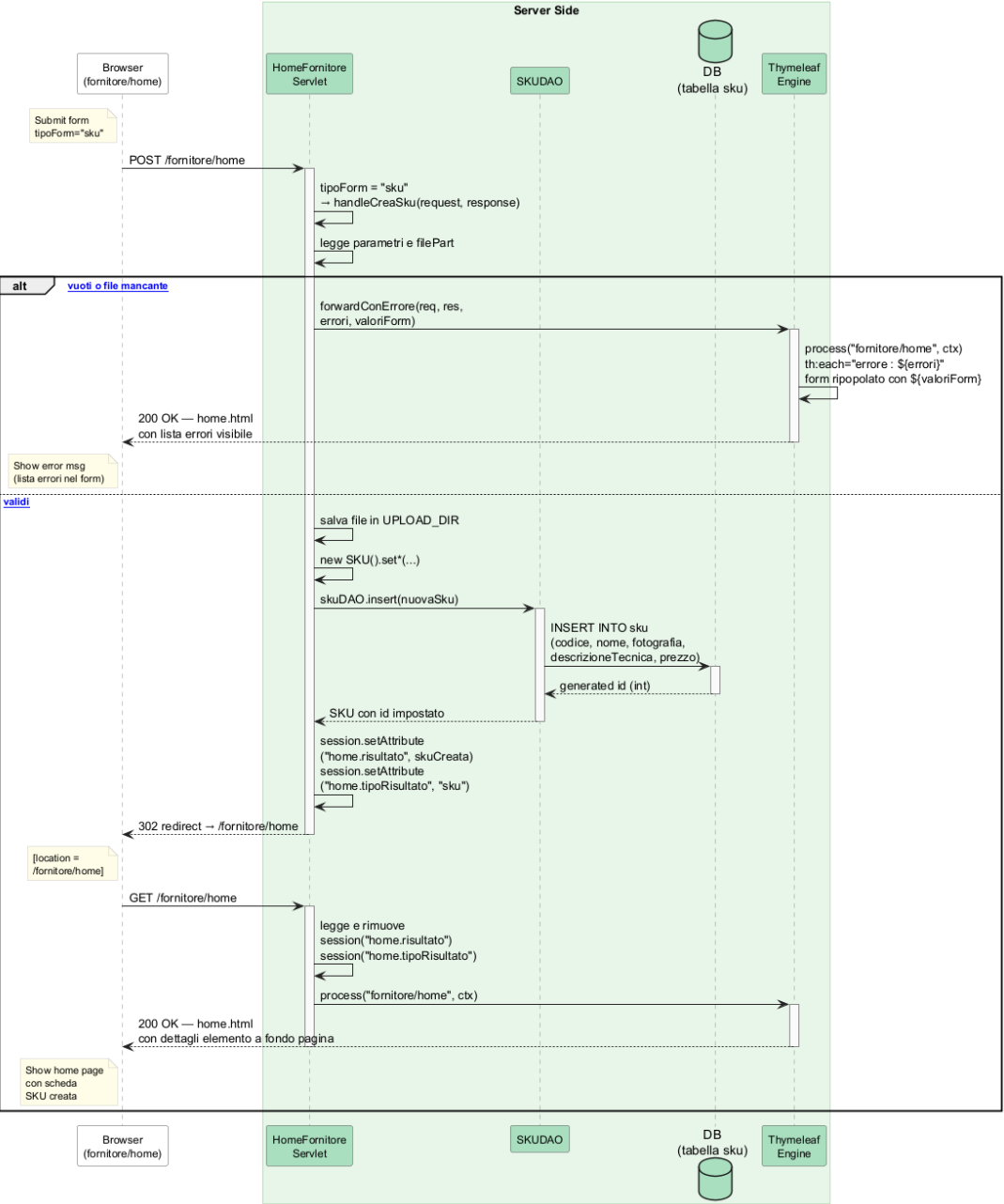
Evento: Login (SPA)

Colore	Lato
	Client side
	Server side



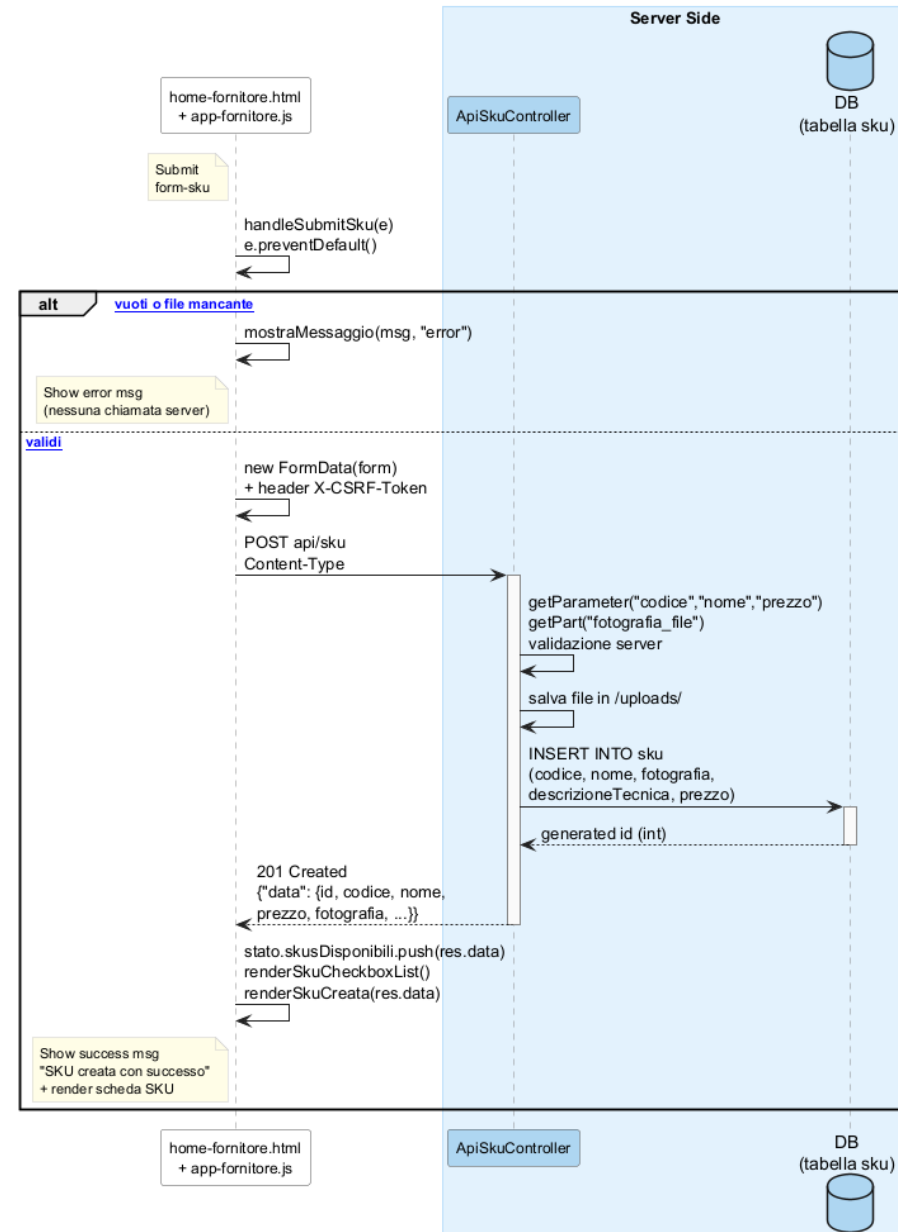
Evento: Creazione SKU (SSR — Thymeleaf)

Colore	Lato
	Client side
	Server side



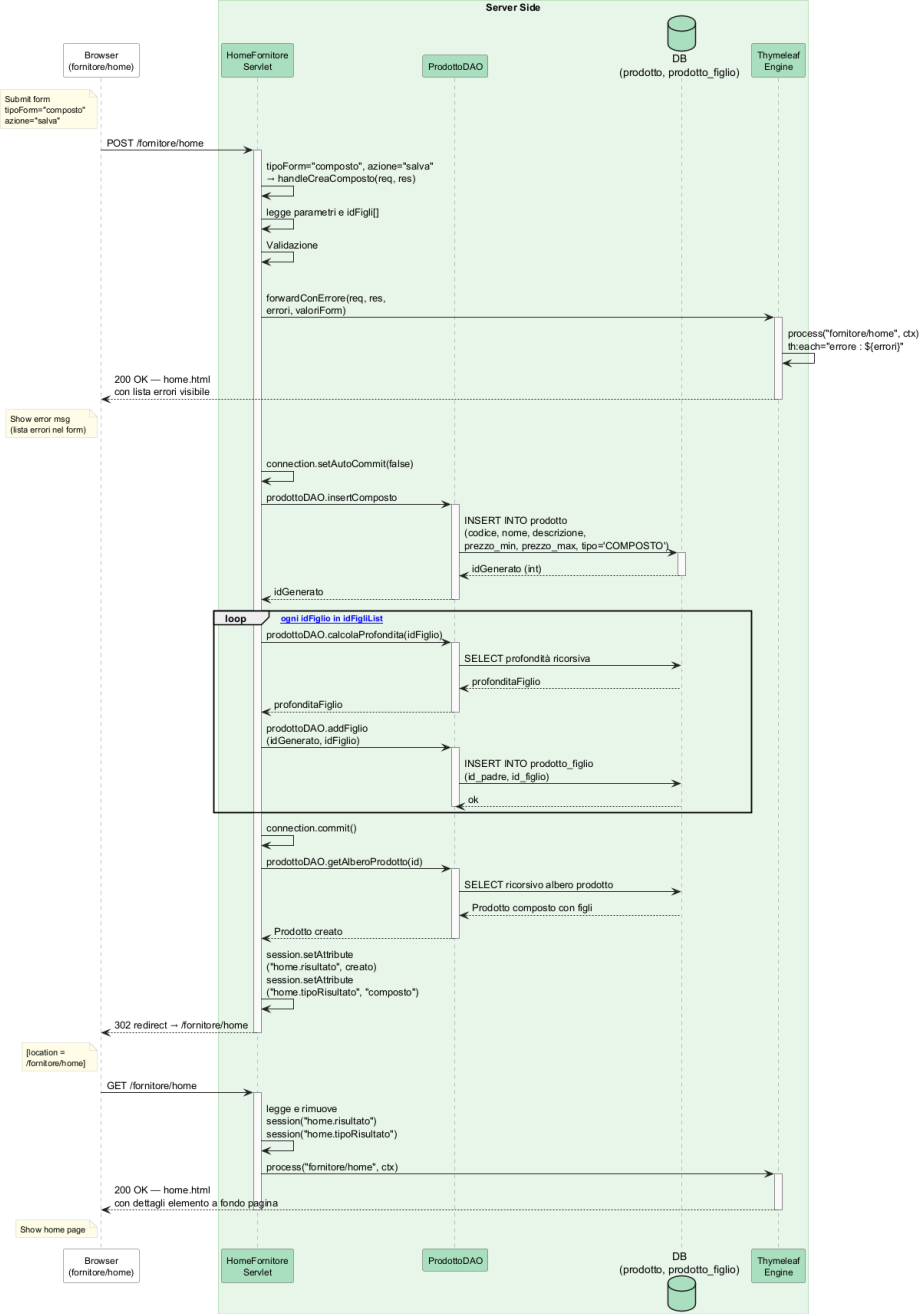
Evento: Creazione SKU (SPA)

Colore	Lato
	Client side
	Server side



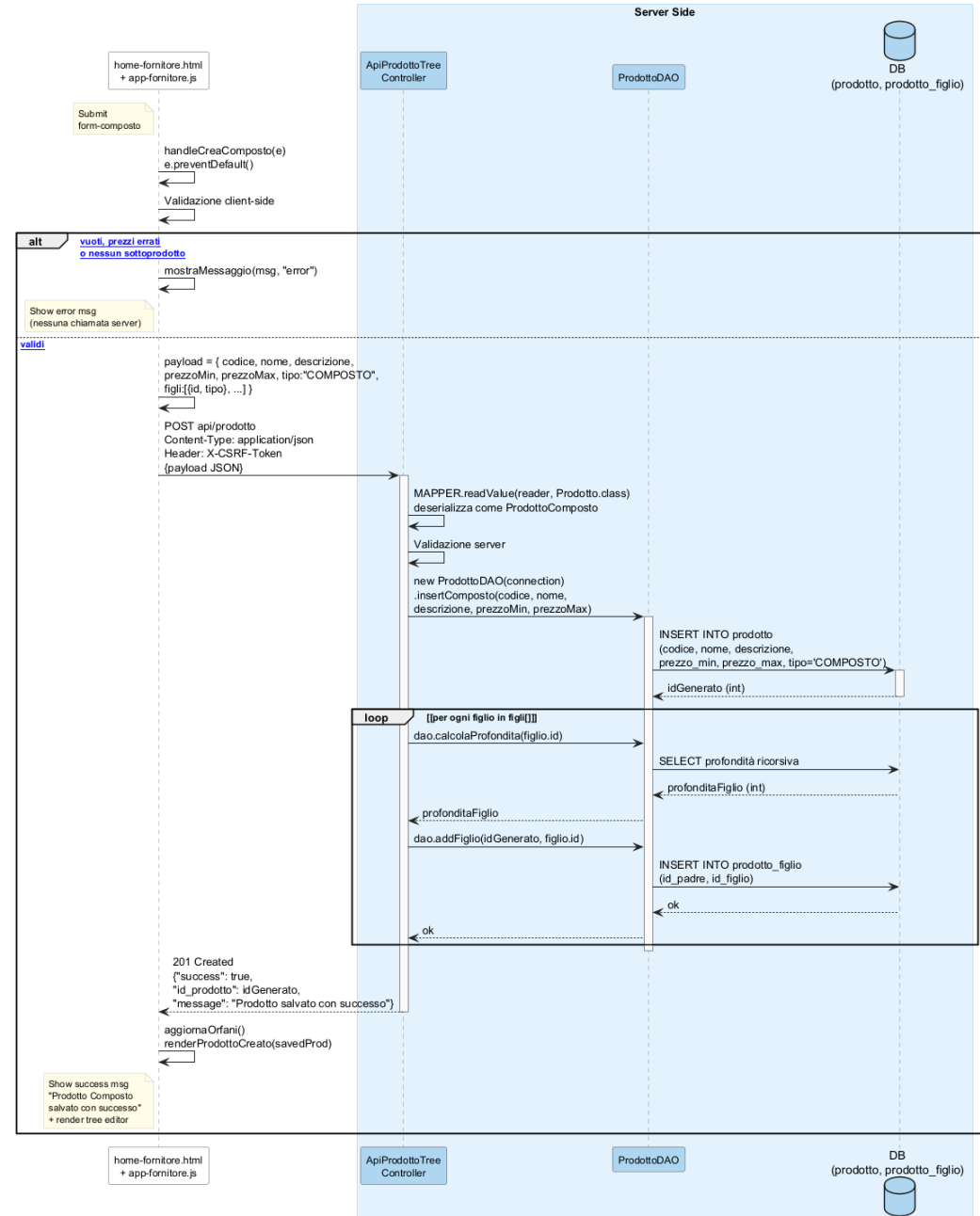
Evento: Creazione Prodotto Composto (SSR — Thymeleaf)

Colore	Lato
Client side	
Server side	

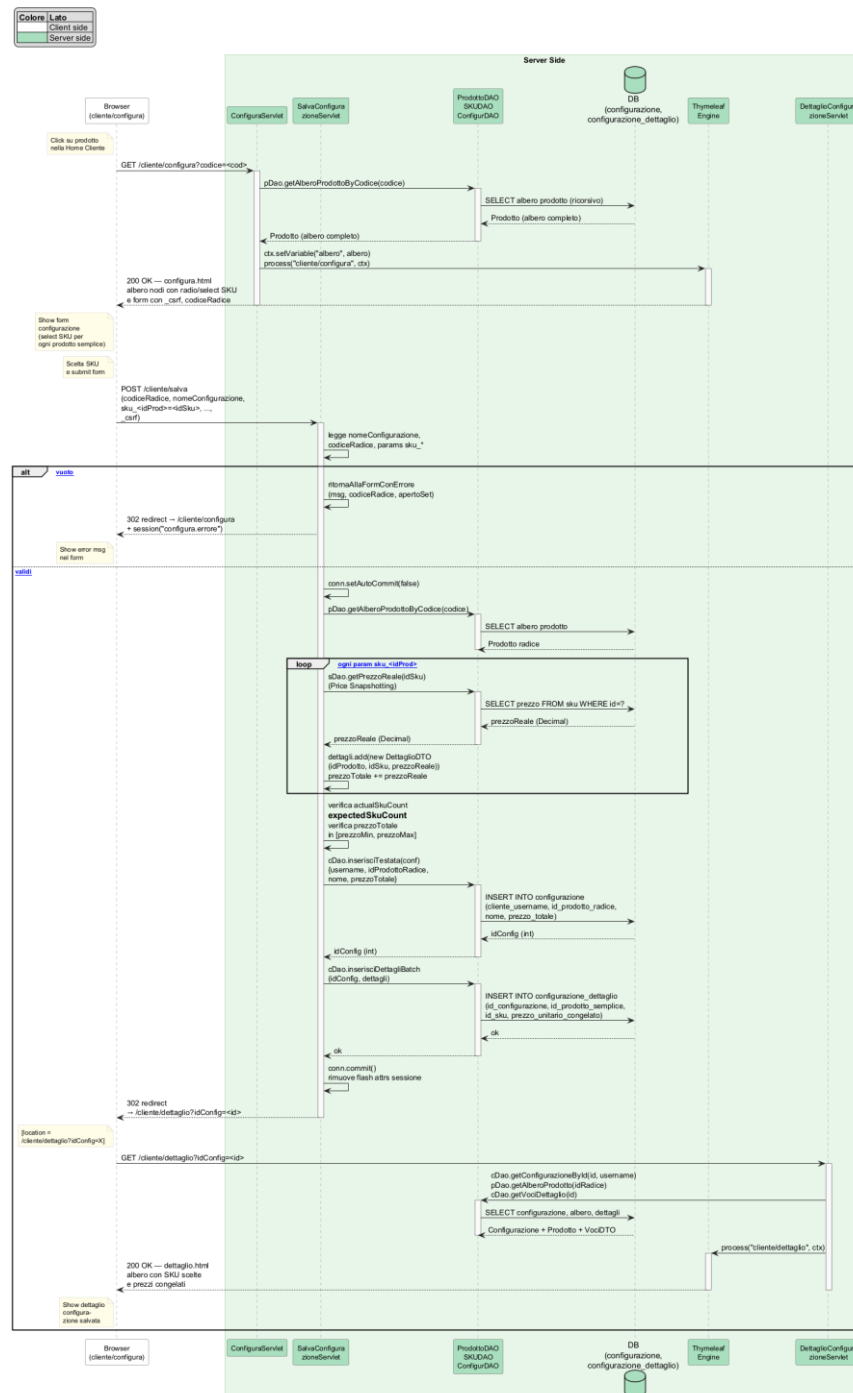


Evento: Creazione Prodotto Composto (SPA)

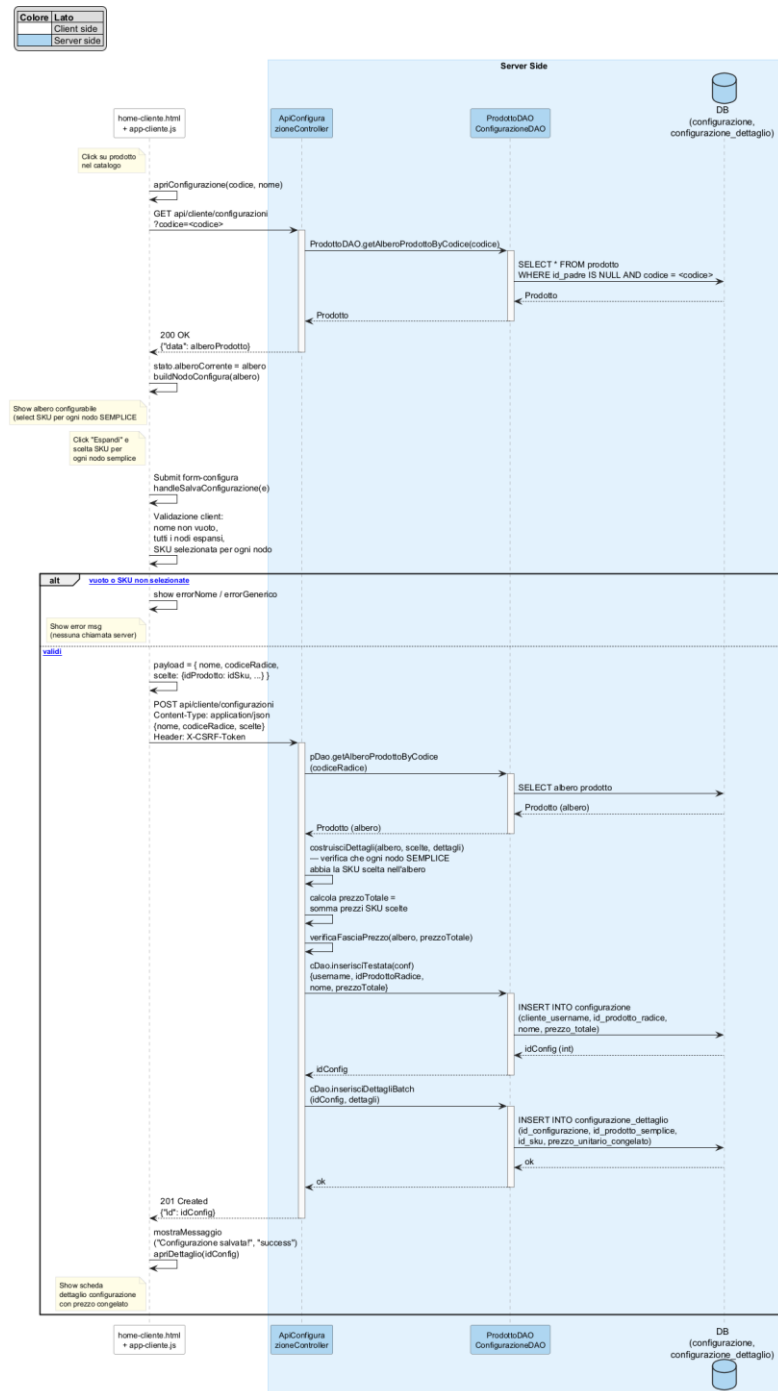
Colore	Lato
	Client side
	Server side



Evento: Creazione Nuova Configurazione (SSR — Thymeleaf)

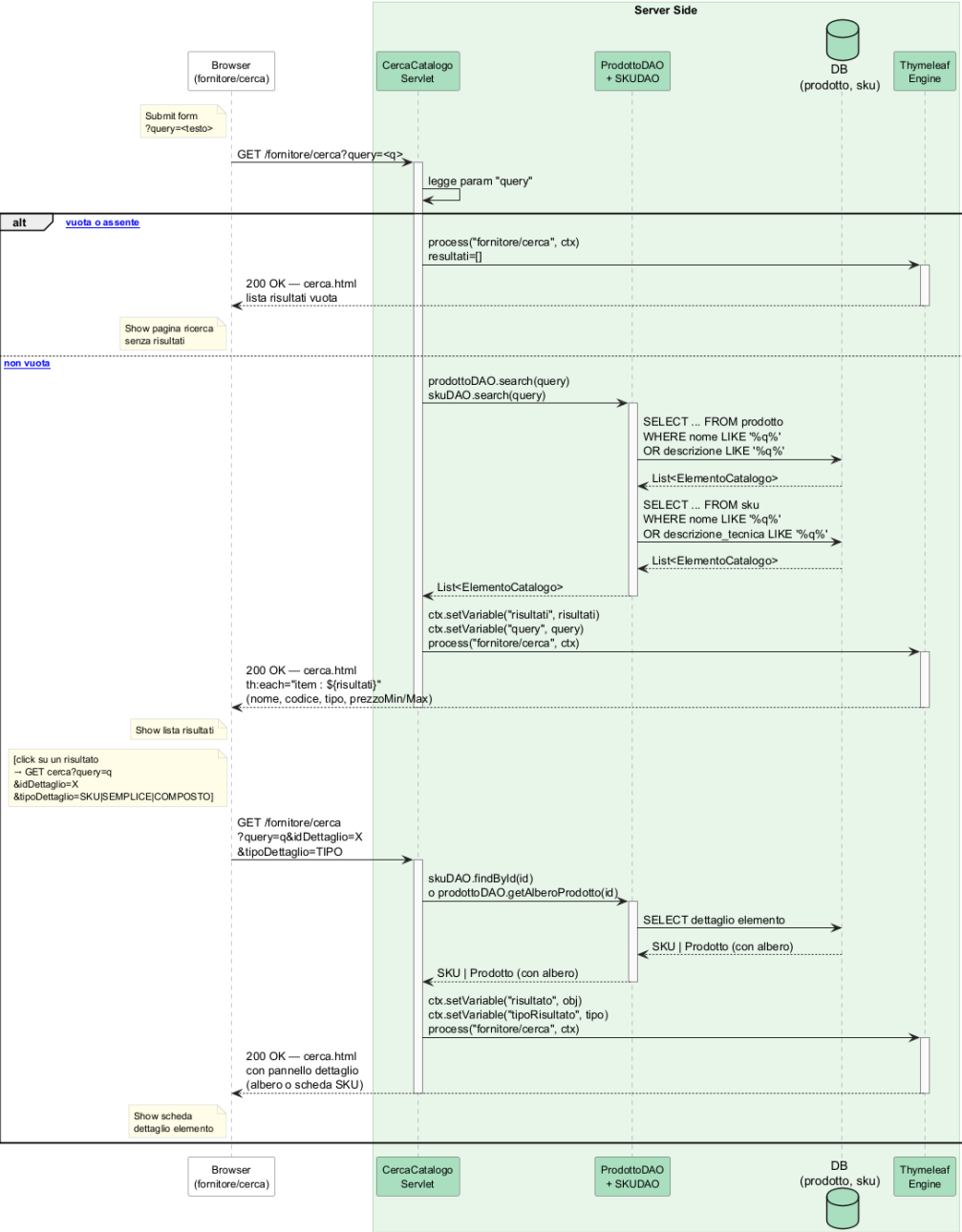


Evento: Creazione Nuova Configurazione (SPA — Cliente)



Evento: Ricerca Catalogo (SSR — Thymeleaf)

Colore	Lato
	Client side
	Server side



Evento: Ricerca Catalogo (SPA — Fornitore)

Colore	Lato
	Client side
	Server side

